

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**NGUYỄN THỊ MINH HƯƠNG – 52300201
TRẦN TÚ MINH – 52300043
NGUYỄN THỊ TÚ ANH – 52300175**

BÁO CÁO CUỐI KỲ

PHÁT TRIỂN PHẦN MỀM FULL STACK

THÀNH PHỐ HỒ CHÍ MINH, 2026

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**NGUYỄN THỊ MINH HƯƠNG – 52300201
TRẦN TÚ MINH – 52300043
NGUYỄN THỊ TÚ ANH – 52300175**

BÁO CÁO CUỐI KỲ

PHÁT TRIỂN PHẦN MỀM FULL STACK

Người hướng dẫn

ThS. NCS. VŨ ĐÌNH HỒNG

THÀNH PHỐ HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Trước hết, chúng em xin chân thành bày tỏ lòng biết ơn sâu sắc và sự kính trọng đến ThS. NCS. VŨ ĐÌNH HỒNG, người đã trực tiếp hướng dẫn, chỉ bảo và đồng hành cùng chúng em trong suốt quá trình thực hiện báo cáo cuối kỳ này. Thầy không chỉ tận tình truyền đạt những kiến thức chuyên môn quý báu mà còn định hướng, góp ý và hỗ trợ chúng em trong việc tiếp cận, nghiên cứu và giải quyết các vấn đề thực tiễn liên quan đến đề tài.

Tuy nhiên, do kiến thức và kinh nghiệm thực tế của chúng em còn hạn chế, bài báo cáo chắc chắn không tránh khỏi những thiếu sót và hạn chế nhất định. Chúng em rất mong nhận được sự thông cảm cũng như những ý kiến đóng góp quý báu từ thầy để chúng em có thể hoàn thiện hơn trong tương lai.

Một lần nữa, chúng em xin kính chúc thầy luôn dồi dào sức khỏe, hạnh phúc và ngày càng thành công trong sự nghiệp giảng dạy cũng như nghiên cứu khoa học.

Chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, Ngày 18 tháng 03 năm 2026

Tác giả

(Ký và ghi rõ họ tên)

Nguyễn Thị Minh Hương

Nguyễn Thị Tú Anh

Trần Tú Minh

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Nhóm chúng em xin cam đoan đây là công trình nghiên cứu của riêng chúng em và được sự hướng dẫn khoa học của ThS. NCS. VŨ ĐÌNH HỒNG. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong báo cáo còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào nhóm chúng em xin hoàn toàn chịu trách nhiệm về nội dung Báo cáo của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do chúng em gây ra trong quá trình thực hiện (nếu có).

TP.Hồ Chí Minh, Ngày 18 tháng 03 năm 2026

Tác giả

(ký và ghi rõ họ tên)

Nguyễn Thị Minh Hương

Nguyễn Thị Tú Anh

Trần Tú Minh

TÓM TẮT

Báo cáo gồm 7 chương:

- **Chương 1 - Tổng quan đề tài**

Trình bày lý do chọn đề tài, mục tiêu xây dựng hệ thống giao tiếp trực tuyến và các yêu cầu đặt ra về tính năng cũng như khả năng mở rộng.

- **Chương 2 - Cơ sở lý thuyết**

Giới thiệu các công nghệ và kiến thức nền tảng sử dụng trong hệ thống như WebSocket, Socket.io, WebRTC, MongoDB, Redis, Docker và mô hình Client–Server.

- **Chương 3 - Kiến trúc hệ thống**

Mô tả tổng thể kiến trúc của hệ thống, bao gồm các thành phần frontend, backend, database, cache và reverse proxy. Trình bày sơ đồ luồng dữ liệu và cách các thành phần tương tác với nhau.

- **Chương 4 - Xây dựng và phát triển hệ thống**

Trình bày quá trình cài đặt và triển khai các chức năng chính như xác thực người dùng, chat 1-1, chat nhóm, gửi file và hiển thị trạng thái thời gian thực.

- **Chương 5 - Tích hợp gọi điện WebRTC**

Mô tả cơ chế hoạt động của WebRTC, cách thiết lập kết nối giữa hai client thông qua signaling server và triển khai chức năng gọi Audio/Video.

- **Chương 6 - Tối ưu hiệu năng và khả năng mở rộng**

Trình bày các giải pháp nâng cao hiệu năng hệ thống như sử dụng Redis để cache và Pub/Sub, triển khai nhiều backend instance, và sử dụng Nginx để cân bằng tải.

- **Chương 7 - Kết luận**

Tổng kết kết quả đạt được, đánh giá ưu điểm, hạn chế của hệ thống và đề xuất hướng phát triển trong tương lai.

MỤC LỤC

TÓM TẮT

DANH MỤC HÌNH VẼ	i
------------------	---

DANH MỤC BẢNG BIỂU	ii
--------------------	----

CHƯƠNG 1 - TỔNG QUAN ĐỀ TÀI	1
-----------------------------	---

1.1 Giới thiệu đề tài	1
1.2 Mục tiêu của đề tài	1
1.3 Phạm vi nghiên cứu	1
1.4 Phương pháp thực hiện	2
1.5 Ý nghĩa của đề tài	2

CHƯƠNG 2 - CƠ SỞ LÝ THUYẾT	3
----------------------------	---

2.1 Kiến trúc Client – Server	3
2.2 Giao tiếp thời gian thực với WebSocket	3
2.3 Thư viện Socket.io	4
2.4 Cơ sở dữ liệu MongoDB	4
2.5 Redis và cơ chế Pub/Sub	5
2.6 Giao thức WebRTC	6
2.7 Docker và Docker Compose	6
2.8 Nginx và cân bằng tải	7

CHƯƠNG 3 - KIẾN TRÚC HỆ THỐNG	8
3.1 Sơ đồ Use Case tổng quát	8
3.2 Tổng quan kiến trúc hệ thống	8
3.3 Sơ đồ kiến trúc hệ thống	9
3.4 Các thành phần trong hệ thống	9
3.4.1 Frontend	9
3.4.2 Backend	10
3.4.3 Cơ sở dữ liệu MongoDB	10
3.4.4 Redis	10
3.4.5 Nginx	11
3.5 Luồng hoạt động của hệ thống	11
3.5.1 Luồng nhắn tin thời gian thực	11
3.5.2 Luồng gọi video WebRTC	11
3.5.3 Luồng cân bằng tải	12
3.6 Khả năng mở rộng của hệ thống	12
3.7 Triển khai hệ thống bằng Docker	12
3.8 Đánh giá kiến trúc	13
 CHƯƠNG 4 - XÂY DỰNG VÀ PHÁT TRIỂN HỆ THỐNG	 14
4.1 Thiết kế cơ sở dữ liệu	14
4.1.1 Tổng quan schema và sơ đồ ERD	14
4.1.2 Collection users	14
4.1.3 Collection conversations	15

4.1.4	Collection messages	17
4.1.5	Các collection phụ trợ	17
4.2	Xác thực người dùng	18
4.2.1	Luồng đăng ký và xác thực email	18
4.2.2	Luồng đăng nhập và sinh JWT	18
4.2.3	Middleware xác thực và phân quyền	19
4.2.4	Đặt lại mật khẩu	19
4.3	Quản lý danh bạ và bạn bè	19
4.3.1	Luồng kết bạn	19
4.3.2	Cơ chế chặn người dùng	20
4.4	Hệ thống nhắn tin thời gian thực	21
4.4.1	Kiến trúc Socket.io và phòng chat	21
4.4.2	Xử lý sự kiện gửi tin nhắn	21
4.4.3	Các tính năng nhắn tin nâng cao	22
4.4.4	Quản lý nhóm chat	22
4.5	API REST và xử lý tin nhắn	23
4.5.1	Phân trang và lọc tin nhắn	23
4.5.2	Tìm kiếm tin nhắn	23
4.5.3	Upload file	24
4.6	Đếm tin nhắn chưa đọc	24
CHƯƠNG 5 - TÍCH HỢP GỌI ĐIỆN WEBRTC		25
5.1	Tổng quan WebRTC và vai trò của Signaling Server	25

5.2	Kiến trúc hệ thống gọi điện phía server	27
5.2.1	Quản lý trạng thái cuộc gọi trong bộ nhớ	27
5.2.2	Thiết kế Signaling Server	27
5.2.3	Cơ chế Grace Period khi mất kết nối	28
5.3	Luồng thiết lập cuộc gọi 1-1	29
5.3.1	Phía người gọi (Caller)	29
5.3.2	Phía người nhận (Callee)	30
5.3.3	Trao đổi ICE Candidate	30
5.3.4	Nhận luồng media từ peer	30
5.4	Kiến trúc frontend WebRTC	31
5.4.1	Phân tách trách nhiệm giữa các module	31
5.4.2	Custom Hook useWebRTC	31
5.4.3	Quản lý trạng thái cuộc gọi trong CallManager	32
5.5	Các tính năng nâng cao trong cuộc gọi	32
5.5.1	Voice Activity Detection (VAD)	32
5.5.2	Điều khiển media	33
5.5.3	Chia sẻ màn hình (Screen Share)	33
5.5.4	Tóm tắt cuộc gọi và thông báo hệ thống	34
5.6	Luồng gọi nhóm	34
5.7	Xử lý lỗi và độ bền của cuộc gọi	35
5.7.1	Các trường hợp lỗi media	35
5.7.2	Xử lý cuộc gọi bận	35
5.7.3	Thống kê kết nối	36

5.7.4	Cơ chế lưu tạm ICE Candidate	36
5.8	Luồng Socket.io tổng thể của hệ thống	36

CHƯƠNG 6 - GIẢI PHÁP TỐI ƯU HIỆU NĂNG, MỞ RỘNG HỆ THỐNG VÀ BẢO MẬT TOÀN DIỆN **39**

6.1	Các giải pháp đã triển khai trong hệ thống	39
6.1.1	Redis – Caching và Pub/Sub phân tán	39
6.1.2	Kiến trúc Stateless và Horizontal Scaling	41
6.1.3	Nginx – Reverse Proxy và Load Balancing	41
6.1.4	Tối ưu MongoDB	42
6.1.5	Cơ chế xác thực và bảo vệ cơ bản	43
6.2	Các đề xuất nâng cấp cho môi trường production	44
6.2.1	Xác thực đa yếu tố (MFA/TOTP)	44
6.2.2	Kiến trúc Access Token / Refresh Token	44
6.2.3	Tăng cường bảo vệ hệ thống	44
6.2.4	Giám sát và phát hiện tấn công	45
6.2.5	Quy trình ứng phó sự cố	45
6.3	Phân tích kiến trúc và trade-off tổng hợp	45

CHƯƠNG 7 - KẾ HOẠCH TRIỂN KHAI DEMO CUỐI KỲ **47**

7.1	Mục tiêu	47
7.2	Công cụ sử dụng (Tools Required)	47
7.3	Kịch bản Demo (Scenario)	48
7.4	Kế hoạch triển khai (Deployment Plan)	53

7.5	Tiêu chí đánh giá thành công (Success Criteria)	54
7.6	Kết quả mong đợi (Expected Results)	56

TÀI LIỆU THAM KHẢO	57
---------------------------	-----------

DANH MỤC HÌNH VẼ

Hình 3.1	Sơ đồ Use Case tổng quát của hệ thống	8
Hình 3.2	Sơ đồ kiến trúc tổng thể hệ thống	9
Hình 4.1	Sơ đồ ERD của hệ thống nhắn tin	14
Hình 5.1	Sơ đồ luồng thiết lập cuộc gọi WebRTC qua Socket.io	26
Hình 5.2	Sơ đồ luồng Socket.io tổng thể của hệ thống	37
Hình 6.1	Redis monitor ghi nhận các lệnh PUBLISH khi người dùng gửi tin nhắn – minh chứng Redis Pub/Sub đồng bộ sự kiện Socket.io theo thời gian thực	40
Hình 6.2	Log Nginx ghi nhận các request được proxy tới backend – minh chứng Nginx hoạt động như reverse proxy phân phối traffic	42
Hình 6.3	Log backend ghi nhận kết nối WebSocket, xác thực JWT và xử lý API request theo thời gian thực	43

DANH MỤC BẢNG BIỂU

Bảng 4.1	Cấu trúc collection users	15
Bảng 4.2	Cấu trúc collection conversations	16
Bảng 4.3	Cấu trúc collection messages	17
Bảng 5.1	Bảng sự kiện Socket.io trong hệ thống gọi điện WebRTC	28
Bảng 5.2	Phân tách trách nhiệm giữa các module WebRTC phía client . .	31
Bảng 5.3	Ánh xạ lỗi media WebRTC sang thông báo người dùng	35
Bảng 6.1	Các biện pháp tăng cường bảo vệ hệ thống	44
Bảng 6.2	Tổng hợp trade-off các giải pháp kiến trúc	45

CHƯƠNG 1 - TỔNG QUAN ĐỀ TÀI

1.1 Giới thiệu đề tài

Trong thời đại công nghệ số hiện nay, nhu cầu giao tiếp trực tuyến ngày càng gia tăng, đặc biệt trong bối cảnh làm việc từ xa và học tập online trở nên phổ biến. Các nền tảng như Zalo, Discord hay Slack đã chứng minh được vai trò quan trọng trong việc kết nối người dùng thông qua các tính năng nhắn tin, gọi điện và chia sẻ dữ liệu theo thời gian thực.

Xuất phát từ nhu cầu thực tế đó, đề tài hướng đến việc xây dựng một hệ thống giao tiếp trực tuyến hoạt động trên nền web, cho phép người dùng tương tác nhanh chóng, tiện lợi và hiệu quả. Hệ thống không chỉ dừng lại ở chức năng nhắn tin cơ bản mà còn tích hợp nhiều tính năng nâng cao như chat nhóm, gửi file và gọi Audio/Video.

1.2 Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng một nền tảng chat trực tuyến hoàn chỉnh với các chức năng:

- Xây dựng hệ thống nhắn tin thời gian thực (real-time) giữa các người dùng.
- Hỗ trợ tạo nhóm chat và quản lý thành viên trong nhóm.
- Cho phép gửi hình ảnh và file đính kèm.
- Hiển thị trạng thái người dùng (online/offline, đang nhập tin nhắn).
- Tích hợp chức năng gọi điện Audio/Video sử dụng WebRTC.
- Đảm bảo hệ thống có khả năng mở rộng và hoạt động ổn định khi có nhiều người dùng truy cập đồng thời.

1.3 Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu và triển khai các nội dung sau:

- Xây dựng frontend cho ứng dụng chat trên nền web.
- Xây dựng backend xử lý logic nghiệp vụ và giao tiếp thời gian thực.
- Thiết kế và triển khai cơ sở dữ liệu lưu trữ thông tin người dùng và tin nhắn.
- Tích hợp công nghệ WebRTC để thực hiện cuộc gọi giữa các client.
- Triển khai hệ thống bằng Docker và Docker Compose.
- Tối ưu hiệu năng và khả năng mở rộng thông qua Redis và Nginx.

1.4 Phương pháp thực hiện

Để thực hiện đề tài, nhóm áp dụng mô hình phát triển hệ thống theo kiến trúc Client–Server. Các công nghệ chính được sử dụng bao gồm:

- Frontend: ReactJS để xây dựng giao diện người dùng.
- Backend: NodeJS kết hợp ExpressJS và Socket.io để xử lý kết nối thời gian thực.
- Cơ sở dữ liệu: MongoDB để lưu trữ dữ liệu tin nhắn và người dùng.
- Cache và đồng bộ dữ liệu: Redis sử dụng cho cơ chế cache và Pub/Sub.
- Giao thức gọi điện: WebRTC cho phép truyền tải âm thanh và hình ảnh theo thời gian thực.
- Triển khai hệ thống: Docker, Docker Compose và Nginx.

1.5 Ý nghĩa của đề tài

Đề tài không chỉ giúp hiểu rõ cách xây dựng một hệ thống chat thời gian thực mà còn cung cấp kiến thức về thiết kế hệ thống có khả năng mở rộng và tối ưu hiệu năng. Đây là những kỹ năng quan trọng trong lĩnh vực phát triển phần mềm hiện đại, đặc biệt đối với các hệ thống phục vụ lượng lớn người dùng.

Ngoài ra, kết quả của đề tài có thể được phát triển và mở rộng thành các ứng dụng thực tế trong doanh nghiệp hoặc các nền tảng giao tiếp trực tuyến quy mô lớn.

CHƯƠNG 2 - CƠ SỞ LÝ THUYẾT

2.1 Kiến trúc Client – Server

Kiến trúc Client – Server là một mô hình thiết kế hệ thống trong đó các thành phần được phân tách thành hai nhóm chính: client và server. Client là phía người dùng cuối (trình duyệt web hoặc ứng dụng), chịu trách nhiệm gửi yêu cầu và hiển thị dữ liệu. Server là nơi xử lý các yêu cầu đó, thực hiện logic nghiệp vụ và trả về kết quả.

Trong hệ thống chat trực tuyến, kiến trúc này được áp dụng để đảm bảo sự phân tách rõ ràng giữa giao diện người dùng và xử lý dữ liệu. Client sử dụng các API hoặc kết nối WebSocket để giao tiếp với server. Server chịu trách nhiệm quản lý phiên đăng nhập, xử lý tin nhắn, đồng bộ dữ liệu và lưu trữ thông tin vào cơ sở dữ liệu.

Ưu điểm của kiến trúc này là dễ mở rộng, dễ bảo trì và phù hợp với các hệ thống có nhiều người dùng truy cập đồng thời.

2.2 Giao tiếp thời gian thực với WebSocket

WebSocket là một giao thức truyền thông cho phép thiết lập kết nối hai chiều (full-duplex) giữa client và server thông qua một kết nối TCP duy nhất. Sau khi kết nối được thiết lập, cả hai phía đều có thể chủ động gửi dữ liệu mà không cần chờ yêu cầu từ phía còn lại.

So với giao thức HTTP truyền thông, WebSocket có những ưu điểm nổi bật:

- Giảm độ trễ do không cần thiết lập lại kết nối cho mỗi lần gửi dữ liệu.
- Cho phép truyền dữ liệu liên tục theo thời gian thực.
- Tiết kiệm băng thông do giảm overhead của HTTP headers.

Trong hệ thống chat, WebSocket đóng vai trò cốt lõi giúp đảm bảo tin nhắn được truyền ngay lập tức giữa các người dùng. Khi một người gửi tin nhắn, server sẽ nhận

dữ liệu và ngay lập tức đẩy đến người nhận thông qua kết nối đã được thiết lập trước đó.

2.3 Thư viện Socket.io

Socket.io là một thư viện JavaScript giúp đơn giản hóa việc xây dựng ứng dụng realtime dựa trên WebSocket. Ngoài việc hỗ trợ WebSocket, Socket.io còn có khả năng fallback sang các phương thức khác như polling khi trình duyệt không hỗ trợ.

Các tính năng nổi bật của Socket.io bao gồm:

- Tự động kết nối lại (auto-reconnect) khi mất kết nối.
- Hỗ trợ broadcast tin nhắn đến nhiều client.
- Quản lý phòng (room) và namespace để tổ chức các nhóm người dùng.
- Đồng bộ sự kiện giữa client và server.

Trong đề tài, Socket.io được sử dụng để xây dựng các chức năng:

- Chat 1-1 giữa hai người dùng.
- Chat nhóm thông qua cơ chế room.
- Hiển thị trạng thái “đang nhập” (typing).
- Cập nhật trạng thái online/offline theo thời gian thực.

2.4 Cơ sở dữ liệu MongoDB

MongoDB là hệ quản trị cơ sở dữ liệu NoSQL sử dụng mô hình document-oriented, trong đó dữ liệu được lưu trữ dưới dạng BSON (Binary JSON). Điều này giúp dữ liệu linh hoạt hơn so với mô hình bảng của cơ sở dữ liệu quan hệ.

Ưu điểm của MongoDB:

- Hiệu năng cao trong việc ghi và đọc dữ liệu lớn.
- Dễ dàng mở rộng theo chiều ngang (horizontal scaling).

- Phù hợp với dữ liệu phi cấu trúc hoặc thay đổi thường xuyên.

Trong hệ thống chat, MongoDB được sử dụng để lưu trữ:

- Thông tin người dùng (User Profile).
- Danh sách bạn bè và quan hệ giữa người dùng.
- Lịch sử tin nhắn (Message Logs).

Việc lưu trữ tin nhắn dưới dạng document giúp dễ dàng truy xuất theo từng cuộc trò chuyện và tối ưu hiệu năng khi hệ thống có lượng dữ liệu lớn.

2.5 Redis và cơ chế Pub/Sub

Redis là một hệ quản trị dữ liệu dạng key-value lưu trữ trên bộ nhớ (in-memory), cho tốc độ truy xuất cực nhanh. Redis thường được sử dụng cho caching, session storage và message broker.

Một trong những tính năng quan trọng của Redis là cơ chế Publish/Subscribe (Pub/Sub), cho phép các thành phần trong hệ thống giao tiếp với nhau thông qua các kênh (channel).

Cơ chế hoạt động:

- Một server publish (gửi) thông điệp lên một channel.
- Các server khác subscribe (đăng ký) channel đó sẽ nhận được thông điệp ngay lập tức.

Trong hệ thống chat, Redis được sử dụng để:

- Đồng bộ tin nhắn giữa nhiều instance backend khi triển khai scale.
- Cache dữ liệu truy xuất thường xuyên để giảm tải cho database.

Điều này giúp hệ thống hoạt động ổn định ngay cả khi có nhiều server chạy song song.

2.6 Giao thức WebRTC

WebRTC (Web Real-Time Communication) là một công nghệ cho phép truyền tải âm thanh, video và dữ liệu trực tiếp giữa các trình duyệt mà không cần thông qua server trung gian.

Quá trình thiết lập kết nối WebRTC gồm các bước:

- Trao đổi thông tin kết nối (SDP) thông qua signaling server.
- Thiết lập kết nối ngang hàng (Peer-to-Peer).
- Truyền dữ liệu media trực tiếp giữa hai client.

Ưu điểm của WebRTC:

- Độ trễ thấp do truyền dữ liệu trực tiếp.
- Giảm tải cho server.
- Hỗ trợ truyền tải audio, video chất lượng cao.

Trong đề tài, WebRTC được sử dụng để triển khai chức năng gọi Audio/Video giữa hai người dùng, kết hợp với Socket.io để thực hiện signaling.

2.7 Docker và Docker Compose

Docker là nền tảng giúp đóng gói ứng dụng cùng với toàn bộ môi trường chạy vào các container độc lập. Điều này giúp đảm bảo ứng dụng hoạt động nhất quán trên mọi môi trường.

Docker Compose là công cụ cho phép định nghĩa và quản lý nhiều container thông qua một file cấu hình duy nhất (docker-compose.yml).

Trong đề tài, Docker được sử dụng để:

- Đóng gói các thành phần của hệ thống: frontend, backend, database, cache.
- Triển khai hệ thống nhanh chóng chỉ với một lệnh.

- Dễ dàng mở rộng số lượng container khi cần scale hệ thống.

2.8 Nginx và cân bằng tải

Nginx là một web server mạnh mẽ, đồng thời có thể hoạt động như một Reverse Proxy và Load Balancer. Khi hệ thống có nhiều backend instance, Nginx sẽ phân phối request đến các server này theo các thuật toán như round-robin.

Vai trò của Nginx trong hệ thống:

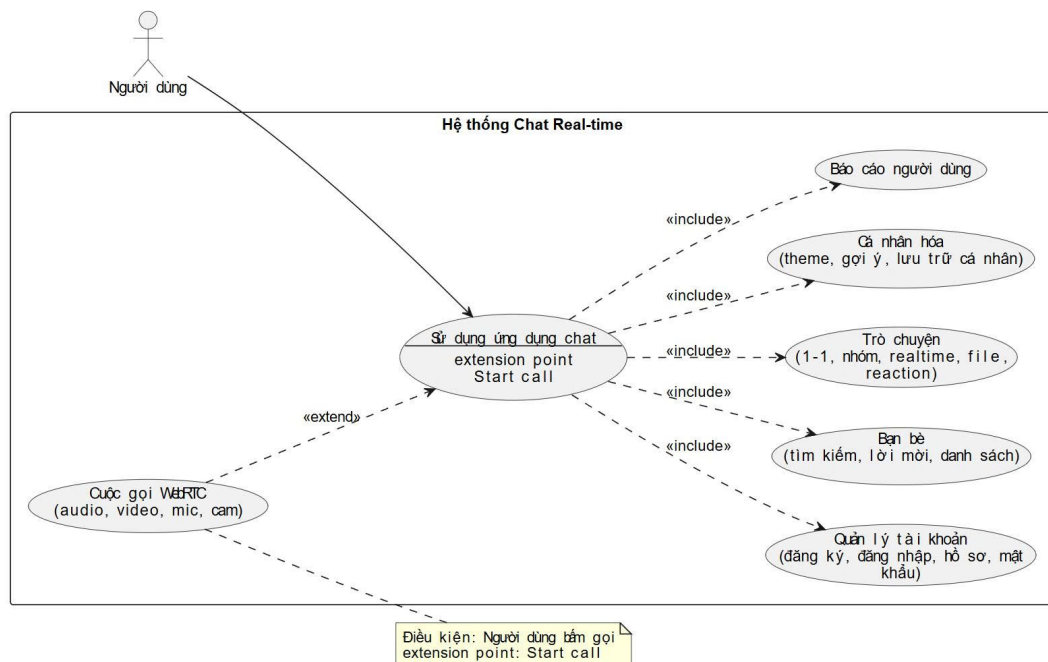
- Là cổng truy cập chính cho người dùng.
- Phân phối request đến các backend server.
- Hỗ trợ cân bằng tải và tăng khả năng chịu lỗi.

Việc sử dụng Nginx kết hợp với Redis giúp hệ thống đạt được khả năng mở rộng cao, đảm bảo hoạt động ổn định khi có nhiều người dùng truy cập đồng thời.

CHƯƠNG 3 - KIẾN TRÚC HỆ THỐNG

3.1 Sơ đồ Use Case tổng quát

Sơ đồ Use Case tổng quát mô tả các tác nhân (actor) và các chức năng chính của hệ thống nhắn tin, giúp làm rõ cách người dùng tương tác với hệ thống trước khi đi vào thiết kế kiến trúc.



Hình 3.1 Sơ đồ Use Case tổng quát của hệ thống

3.2 Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình kiến trúc phân tán (Distributed System) dựa trên nền tảng Client – Server. Mục tiêu của kiến trúc là đảm bảo khả năng mở rộng (Scalability), tính sẵn sàng cao (High Availability) và hiệu năng ổn định khi có nhiều người dùng truy cập đồng thời.

Hệ thống bao gồm các thành phần chính:

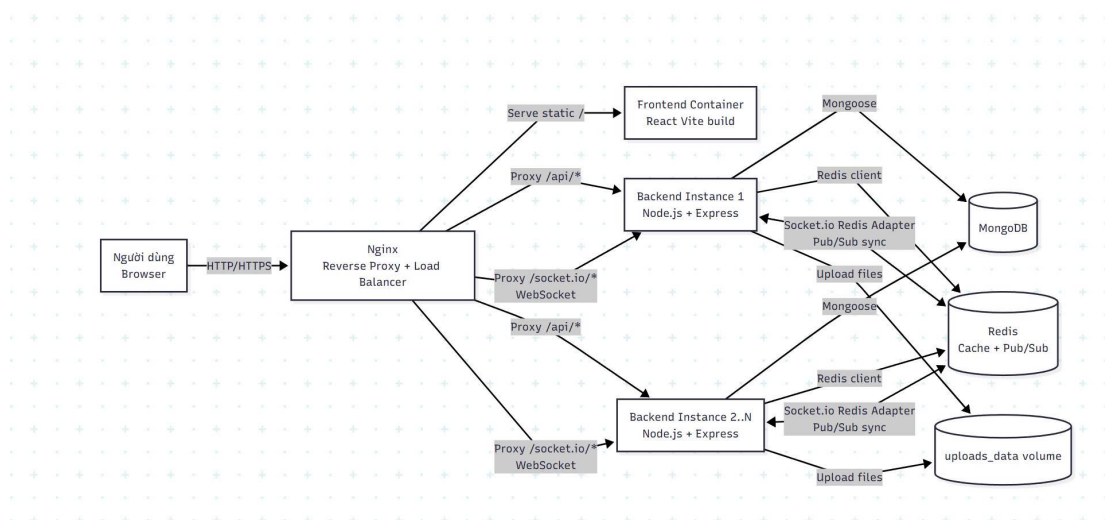
- Frontend (Client)

- Backend (Application Server)
- Database (MongoDB)
- Cache và Message Broker (Redis)
- Reverse Proxy và Load Balancer (Nginx)

Các thành phần này được đóng gói và triển khai dưới dạng container thông qua Docker Compose.

3.3 Sơ đồ kiến trúc hệ thống

Sơ đồ dưới đây mô tả tổng thể kiến trúc hệ thống, bao gồm các thành phần frontend, backend, database, cache và cơ chế cân bằng tải. Các thành phần giao tiếp với nhau thông qua HTTP và WebSocket.



Hình 3.2 Sơ đồ kiến trúc tổng thể hệ thống

3.4 Các thành phần trong hệ thống

3.4.1 Frontend

Frontend được xây dựng bằng ReactJS, chịu trách nhiệm hiển thị giao diện và tương tác với người dùng. Frontend thực hiện các chức năng:

- Gửi yêu cầu HTTP (REST API) đến backend để xử lý các chức năng như đăng nhập, đăng ký.
- Thiết lập kết nối WebSocket thông qua Socket.io để nhận và gửi tin nhắn real-time.
- Hiển thị danh sách bạn bè, tin nhắn và trạng thái người dùng.
- Quản lý trạng thái cuộc gọi video (WebRTC).

3.4.2 *Backend*

Backend được xây dựng bằng NodeJS và ExpressJS, đóng vai trò là trung tâm xử lý logic của hệ thống. Các nhiệm vụ chính bao gồm:

- Xác thực người dùng (JWT Authentication).
- Xử lý các API liên quan đến người dùng và danh bạ.
- Quản lý kết nối WebSocket và điều phối tin nhắn.
- Thực hiện signaling cho WebRTC.

Backend có thể được triển khai nhiều instance để tăng khả năng chịu tải.

3.4.3 *Cơ sở dữ liệu MongoDB*

MongoDB được sử dụng để lưu trữ dữ liệu chính của hệ thống, bao gồm:

- Thông tin người dùng.
- Danh sách bạn bè.
- Lịch sử tin nhắn.

Dữ liệu được tổ chức dưới dạng document giúp việc truy xuất nhanh và linh hoạt.

3.4.4 *Redis*

Redis đóng vai trò là cache và message broker:

- Cache dữ liệu thường xuyên truy xuất.
- Đồng bộ dữ liệu giữa các backend instance thông qua cơ chế Pub/Sub.

Redis giúp đảm bảo hệ thống hoạt động chính xác trong môi trường nhiều server.

3.4.5 Nginx

Nginx được sử dụng như một Reverse Proxy và Load Balancer:

- Nhận request từ client và phân phối đến các backend server.
- Hỗ trợ cân bằng tải (load balancing).
- Tăng hiệu năng và bảo mật hệ thống.

3.5 Luồng hoạt động của hệ thống

3.5.1 Luồng nhắn tin thời gian thực

Quy trình gửi và nhận tin nhắn diễn ra như sau:

1. Người dùng A gửi tin nhắn từ frontend.
2. Tin nhắn được gửi qua WebSocket đến backend server.
3. Backend xử lý và lưu tin nhắn vào MongoDB.
4. Backend phát (emit) tin nhắn đến người dùng B thông qua Socket.io.
5. Nếu hệ thống có nhiều server, Redis sẽ đồng bộ tin nhắn giữa các server.
6. Người dùng B nhận và hiển thị tin nhắn ngay lập tức.

3.5.2 Luồng gọi video WebRTC

Quy trình thiết lập cuộc gọi video:

1. Người dùng A gửi yêu cầu gọi đến người dùng B.

2. Backend đóng vai trò signaling server để trao đổi thông tin kết nối (SDP, ICE candidates).
3. Hai client thiết lập kết nối Peer-to-Peer.
4. Dữ liệu audio/video được truyền trực tiếp giữa hai client.

3.5.3 *Luồng cân bằng tải*

1. Client gửi request đến hệ thống thông qua Nginx.
2. Nginx phân phối request đến một trong các backend instance.
3. Backend xử lý và phản hồi lại client.
4. Redis đảm bảo dữ liệu được đồng bộ giữa các server.

3.6 Khả năng mở rộng của hệ thống

Hệ thống được thiết kế để dễ dàng mở rộng theo chiều ngang (horizontal scaling):

- Tăng số lượng backend instance khi lượng người dùng tăng.
- Sử dụng Redis để đồng bộ dữ liệu giữa các instance.
- Sử dụng Nginx để phân phối tải hiệu quả.

Cách tiếp cận này giúp hệ thống có thể phục vụ hàng nghìn người dùng đồng thời mà không bị quá tải.

3.7 Triển khai hệ thống bằng Docker

Toàn bộ hệ thống được triển khai bằng Docker Compose với các service:

- frontend
- backend (có thể nhiều instance)
- mongodb

- redis
- nginx

Việc sử dụng Docker giúp:

- Đảm bảo môi trường chạy đồng nhất.
- Dễ dàng triển khai và mở rộng.
- Thuận tiện trong việc bảo trì và nâng cấp hệ thống.

3.8 Đánh giá kiến trúc

Kiến trúc hệ thống đáp ứng tốt các yêu cầu của đề tài:

- Hỗ trợ giao tiếp realtime với độ trễ thấp.
- Có khả năng mở rộng khi số lượng người dùng tăng.
- Phân tách rõ ràng giữa các thành phần.
- Dễ dàng triển khai và bảo trì.

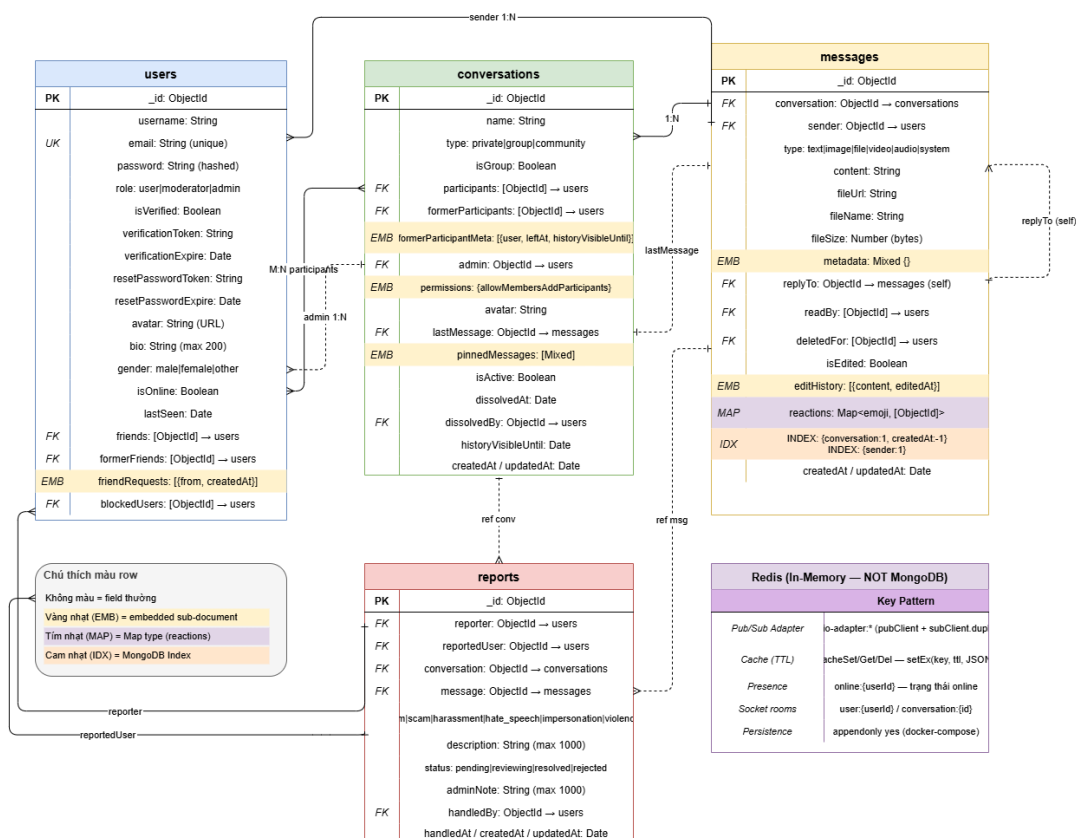
Tuy nhiên, hệ thống vẫn có thể được cải thiện bằng cách bổ sung message queue hoặc triển khai trên môi trường cloud để tăng tính linh hoạt.

CHƯƠNG 4 - XÂY DỰNG VÀ PHÁT TRIỂN HỆ THỐNG

4.1 Thiết kế cơ sở dữ liệu

4.1.1 Tổng quan schema và sơ đồ ERD

Hệ thống sử dụng MongoDB với năm collection chính: users, conversations, messages, reports và personalstorages. Các collection được liên kết với nhau thông qua ObjectId tham chiếu (reference), tuân theo mô hình document NoSQL với chiến lược embedding cho các dữ liệu nhỏ và tham chiếu cho các quan hệ nhiều-nhiều.



Hình 4.1 Sơ đồ ERD của hệ thống nhắn tin

4.1.2 Collection users

Collection users lưu trữ toàn bộ thông tin tài khoản người dùng. Các trường quan trọng bao gồm cơ chế xác thực, quản lý bạn bè và kiểm soát truy cập.

Trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự sinh
username	String	Tên hiển thị, 3–30 ký tự
email	String	Email duy nhất, có index
password	String	Mật khẩu băm bằng bcrypt (salt=10)
role	String (enum)	user moderator admin
isVerified	Boolean	Trạng thái xác thực email
verificationToken	String	Token xác thực email (raw hex)
verificationExpire	Date	Hạn token (24 giờ)
resetPasswordToken	String	Token đặt lại mật khẩu (SHA-256)
resetPasswordExpire	Date	Hạn token đặt lại (30 phút)
isOnline	Boolean	Trạng thái online/offline
lastSeen	Date	Thời điểm hoạt động cuối
friends	[ObjectId]	Mảng tham chiếu tới User (bạn bè)
friendRequests	[{from, createdAt}]	Lời mời kết bạn chờ duyệt
blockedUsers	[ObjectId]	Danh sách người dùng bị chặn
formerFriends	[ObjectId]	Lịch sử bạn bè đã xóa

Bảng 4.1 Cấu trúc collection users

Schema sử dụng Mongoose middleware `pre('save')` để tự động băm mật khẩu bằng bcrypt trước khi lưu. Trường password được đặt `select: false` để không trả về trong các query thông thường, đảm bảo an toàn thông tin.

4.1.3 Collection conversations

Collection conversations hỗ trợ ba loại cuộc trò chuyện: riêng tư (private), nhóm (group) và cộng đồng (community – khi nhóm vượt 300 thành viên). Thiết kế này cho phép hệ thống xử lý cả chat 1-1 và nhóm lớn trên cùng một schema.

Trường	Kiểu dữ liệu	Mô tả
type	String (enum)	private group community
isGroup	Boolean	Cờ phân biệt nhóm và chat riêng tư
participants	[ObjectId]	Thành viên đang hoạt động
formerParticipants	[ObjectId]	Thành viên đã rời/bị kick
formerParticipantMeta	[Object]	Metadata gồm leftAt, historyVisibleUntil
admin	ObjectId	Trưởng nhóm
lastMessage	ObjectId	Ref tới tin nhắn cuối cùng
pinnedMessages	[Mixed]	Tin nhắn được ghim (tối đa 3)
isActive	Boolean	Nhóm còn hoạt động hay không
dissolvedAt	Date	Thời điểm giải tán nhóm
historyVisibleUntil	Date	Cutoff lịch sử cho nhóm đã giải tán

Bảng 4.2 Cấu trúc collection conversations

Trường formerParticipantMeta là điểm thiết kế quan trọng: khi một thành viên rời nhóm hoặc bị kick, hệ thống ghi lại historyVisibleUntil – thời điểm mà người đó chỉ được xem lịch sử đến đó, không thấy tin nhắn mới hơn. Cơ chế này đảm bảo tính riêng tư và nhất quán lịch sử chat.

4.1.4 Collection messages

Trường	Kiểu dữ liệu	Mô tả
conversation	ObjectId	Ref tới Conversation, có index
sender	ObjectId	Ref tới User
type	String (enum)	text image file video audio system
content	String	Nội dung văn bản
fileUrl, fileName, fileSize	String/Number	Thông tin file đính kèm
replyTo	ObjectId	Ref tới Message được trả lời
readBy	[ObjectId]	Danh sách user đã đọc
deletedFor	[ObjectId]	Soft delete per-user
isEdited	Boolean	Cờ đánh dấu đã chỉnh sửa
editHistory	[{content, editedAt}]	Lịch sử các lần sửa
reactions	Map<String, [ObjectId]>	Emoji reaction và danh sách user
metadata	Mixed	Dữ liệu phụ (call info, v.v.)

Bảng 4.3 Cấu trúc collection messages

Trường reactions sử dụng kiểu Map của Mongoose, cho phép lưu trữ động các emoji làm key và mảng userId làm value – phù hợp với yêu cầu thêm/xóa reaction linh hoạt mà không cần thay đổi schema. Index kép {conversation, createdAt: -1} đảm bảo việc phân trang tin nhắn theo thứ tự thời gian đạt hiệu năng cao.

4.1.5 Các collection phụ trợ

Collection reports phục vụ chức năng báo cáo người dùng vi phạm với vòng đời trạng thái pending → reviewing → resolved/rejected do admin xử lý. Collection personalstorages cho phép mỗi user lưu ghi chú cá nhân (tiêu đề, nội dung văn bản, hình ảnh) riêng tư, không chia sẻ với ai.

4.2 Xác thực người dùng

4.2.1 Luồng đăng ký và xác thực email

Hệ thống áp dụng quy trình đăng ký hai bước: tạo tài khoản và xác thực email, nhằm đảm bảo tính hợp lệ của thông tin người dùng trước khi cho phép đăng nhập.

Khi người dùng gửi yêu cầu đăng ký, controller thực hiện tuần tự các bước kiểm tra: độ dài username (tối thiểu 3 ký tự), độ dài mật khẩu (tối thiểu 6 ký tự), tính hợp lệ của giá trị gender, và kiểm tra email trùng lặp trong database. Nếu email đã tồn tại, hệ thống từ chối ngay lập tức để tránh tạo tài khoản trùng.

Sau khi tạo user thành công, hệ thống gọi phương thức `getVerificationToken()` được định nghĩa trên Mongoose model. Phương thức này sinh ngẫu nhiên 32 bytes bằng `crypto.randomBytes()`, lưu token dạng raw hex vào trường `verificationToken` và đặt hạn 24 giờ vào `verificationExpire`. Token này được gửi qua email dưới dạng đường dẫn xác thực.

Khi người dùng nhấn vào link xác thực, server tra cứu user theo token và kiểm tra `verificationExpire > Date.now()`. Nếu hợp lệ, trường `isVerified` được đặt thành `true` và token bị xóa khỏi database. Sau đó người dùng được chuyển hướng về trang đăng nhập với tham số `?verified=true`.

4.2.2 Luồng đăng nhập và sinh JWT

Quá trình đăng nhập kiểm tra tuần tự: email tồn tại, trạng thái `isVerified`, và mật khẩu khớp thông qua `bcrypt.compare()`. Khi xác thực thành công, hệ thống cập nhật `isOnline = true` và `lastSeen`, sau đó sinh JWT bằng hàm `generateToken()`.

JWT được ký bằng `JWT_SECRET` với thời hạn mặc định 7 ngày. Thông tin user được cache vào Redis với TTL 3600 giây (`cacheSet('user:$user._id', user, 3600)`) để giảm tải cho MongoDB trong các request tiếp theo.

4.2.3 Middleware xác thực và phân quyền

Middleware protect được áp dụng cho tất cả các route yêu cầu đăng nhập. Middleware thực hiện ba bước: trích xuất JWT từ header Authorization: Bearer <token>, xác minh chữ ký bằng `jwt.verify()`, và nạp thông tin user.

Để tối ưu hiệu năng, middleware ưu tiên lấy user từ Redis cache trước. Chỉ khi cache miss mới truy vấn MongoDB, sau đó lưu kết quả vào cache. Thông tin user được gắn vào `req.user` để các controller downstream sử dụng.

Middleware `authorize(...roles)` bổ sung kiểm tra phân quyền dựa trên trường `role` của user. Ví dụ, route quản lý báo cáo chỉ cho phép admin và moderator truy cập.

4.2.4 Đặt lại mật khẩu

Khi người dùng yêu cầu đặt lại mật khẩu, phương thức `getResetPasswordToken()` sinh 20 bytes ngẫu nhiên, sau đó lưu phiên bản đã băm SHA-256 vào `resetPasswordToken` (không lưu raw token). Link reset chứa raw token được gửi qua email. Khi xác thực, server băm lại token từ URL bằng SHA-256 rồi so khớp với giá trị trong database – cách tiếp cận này đảm bảo ngay cả khi database bị lộ, kẻ tấn công không thể dùng token đã lưu để đặt lại mật khẩu.

Hàm `forgotPassword` luôn trả về cùng một thông báo thành công dù email có tồn tại hay không, ngăn chặn kỹ thuật user enumeration.

4.3 Quản lý danh bạ và bạn bè

4.3.1 Luồng kết bạn

Hệ thống triển khai quy trình kết bạn hai chiều với đầy đủ các trạng thái: gửi lời mời, chấp nhận, từ chối, hủy lời mời đã gửi và xóa bạn. Mọi thay đổi trạng thái đều được phát (emit) qua Socket.io nhằm cập nhật giao diện theo thời gian thực mà không cần tải lại trang.

Khi người dùng gửi lời mời kết bạn, controller thực hiện các bước kiểm tra tuần tự: không gửi cho chính mình, người nhận có tồn tại, người nhận không chặn người gửi (kiểm tra trường `blockedUsers`), hai bên chưa là bạn và chưa tồn tại lời mời trước đó. Nếu hợp lệ, hệ thống thêm một bản ghi `{from: currentUserId}` vào mảng `friendRequests` của người nhận.

Sau khi lưu vào cơ sở dữ liệu, hệ thống phát hai sự kiện Socket.io:

- `friend_request_received`: gửi tới người nhận để hiển thị thông báo.
- `friend_request_sent`: gửi tới người gửi để đồng bộ giao diện trên các thiết bị.

Khi lời mời được chấp nhận, cả hai phía được cập nhật đồng thời: thêm vào danh sách `friends` của nhau và loại khỏi `formerFriends` (nếu trước đó từng là bạn). Khi xóa bạn, thao tác được thực hiện ngược lại – loại khỏi `friends` và thêm vào `formerFriends` nhằm lưu lại lịch sử.

4.3.2 Cơ chế chặn người dùng

Khi người dùng A chặn người dùng B, hệ thống thực hiện đồng thời nhiều thao tác: thêm B vào danh sách `blockedUsers` của A, xóa B khỏi `friends` của A, đồng thời xóa A khỏi danh sách bạn bè của B và loại bỏ mọi lời mời kết bạn đang chờ giữa hai bên.

Sau đó, các sự kiện Socket.io được phát để cập nhật giao diện hai phía:

- `blocked_by_user`
- `i_blocked_user`
- `friend_removed`

4.4 Hệ thống nhắn tin thời gian thực

4.4.1 Kiến trúc Socket.io và phòng chat

Hệ thống sử dụng hai loại phòng (room) trong Socket.io nhằm phân phối tin nhắn hiệu quả.

Phòng cá nhân `user:{userId}` được tạo ngay khi kết nối socket thành công. Mỗi người dùng luôn tham gia phòng này để nhận các sự kiện cá nhân như thông báo bạn bè hoặc tin nhắn từ các cuộc trò chuyện.

Phòng cuộc trò chuyện `conversation:{conversationId}` được tham gia khi người dùng mở một cuộc trò chuyện cụ thể và rời khỏi khi đóng. Cách tổ chức này giúp giới hạn phạm vi truyền tin nhắn và giảm tải hệ thống.

Chiến lược phân phối tin nhắn được thực hiện theo hai bước:

- Gửi đến phòng cuộc trò chuyện thông qua `conversation:{id}` để cập nhật ngay nội dung chat cho những người đang mở cuộc trò chuyện.
- Đồng thời gửi đến phòng cá nhân `user:{id}` của từng thành viên nhằm cập nhật sidebar và số lượng tin nhắn chưa đọc.

4.4.2 Xử lý sự kiện gửi tin nhắn

Khi nhận sự kiện `send_message`, server thực hiện các bước kiểm tra bảo mật:

- Cuộc trò chuyện phải tồn tại.
- Người gửi là thành viên hợp lệ (`isActiveParticipant`).
- Cuộc trò chuyện chưa bị giải tán (`dissolvedAt`).

Sau khi xác thực thành công, tin nhắn được lưu vào MongoDB kèm theo đầy đủ metadata. Đồng thời, các trường `lastMessage` và `updatedAt` của cuộc trò chuyện được cập nhật để đảm bảo thứ tự hiển thị chính xác trên giao diện.

4.4.3 Các tính năng nhắn tin nâng cao

Hệ thống hỗ trợ nhiều tính năng nhắn tin hiện đại thông qua các sự kiện Socket.io nhằm nâng cao trải nghiệm người dùng.

Trạng thái đang soạn (Typing Indicator): Các sự kiện `typing_start` và `typing_stop` được gửi tới các thành viên trong cùng cuộc trò chuyện. Trước khi chuyển tiếp, server kiểm tra điều kiện `canInteractWithConversation` để đảm bảo chỉ những thành viên hợp lệ mới có thể phát tín hiệu trạng thái.

Đọc tin nhắn (Read Receipt): Sự kiện `message_read` được sử dụng để cập nhật trạng thái đã đọc của các tin nhắn. Hệ thống thực hiện cập nhật hàng loạt bằng phương thức `updateMany()` kết hợp toán tử `$addToSet`, sau đó phát lại kết quả tới cả phòng cuộc trò chuyện và phòng cá nhân của người dùng nhằm đồng bộ trên nhiều thiết bị.

Phản ứng (Reaction): Sự kiện `react_message` cho phép người dùng tương tác với tin nhắn bằng emoji. Nếu người dùng đã phản ứng trước đó thì hệ thống sẽ xóa, ngược lại sẽ thêm mới. Sau khi cập nhật, dữ liệu phản ứng được gửi tới các thành viên trong cuộc trò chuyện để hiển thị theo thời gian thực.

Chỉnh sửa và xóa tin nhắn: Khi chỉnh sửa, nội dung cũ được lưu vào `editHistory` trước khi cập nhật nội dung mới. Khi xóa, hệ thống kiểm tra quyền (chỉ người gửi hoặc quản trị viên nhóm) trước khi xóa khỏi cơ sở dữ liệu và cập nhật lại danh sách tin nhắn được ghim nếu cần.

Ghim tin nhắn: Mỗi cuộc trò chuyện cho phép ghim tối đa một số lượng tin nhắn giới hạn (`PIN_LIMIT`). Khi vượt quá giới hạn, tin nhắn được ghim sớm nhất sẽ bị loại bỏ để đảm bảo số lượng không vượt mức cho phép.

4.4.4 Quản lý nhóm chat

Hệ thống cung cấp đầy đủ các chức năng quản lý nhóm, bao gồm tạo nhóm, thêm và xóa thành viên, chuyển quyền quản trị, rời nhóm và giải tán nhóm.

Khi tạo nhóm, nếu số lượng thành viên vượt quá ngưỡng quy định (COMMUNITY_THRESHOLD), loại nhóm sẽ tự động chuyển từ group sang community nhằm hỗ trợ quy mô lớn hơn.

Khi quản trị viên loại bỏ một thành viên, thời điểm rời nhóm được ghi nhận và sử dụng làm mốc giới hạn hiển thị lịch sử tin nhắn (historyVisibleUntil). Thành viên bị loại chỉ có thể xem các tin nhắn trước thời điểm này.

Cơ chế tương tự cũng được áp dụng khi người dùng tự rời nhóm.

Khi nhóm bị giải tán, hệ thống ghi nhận thời điểm giải tán và áp dụng làm mốc giới hạn lịch sử cho tất cả thành viên đã rời, đảm bảo họ không thể truy cập các tin nhắn phát sinh sau đó.

4.5 API REST và xử lý tin nhắn

4.5.1 Phân trang và lọc tin nhắn

API lấy danh sách tin nhắn hỗ trợ phân trang thông qua các tham số page và limit, với mặc định mỗi trang gồm 50 tin nhắn. Trước khi truy vấn, hệ thống áp dụng cơ chế xác định mốc thời gian hiển thị dựa trên trạng thái tham gia của người dùng, nhằm đảm bảo rằng các thành viên đã rời nhóm không thể xem các tin nhắn được tạo sau thời điểm rời.

Kết quả được sắp xếp theo thứ tự thời gian tăng dần (tin nhắn cũ trước, mới sau). Để tối ưu hiệu năng, hệ thống thực hiện truy vấn theo thứ tự giảm dần, sau đó đảo ngược lại trước khi trả về cho client.

Phản hồi từ server bao gồm dữ liệu tin nhắn và thông tin phân trang như page, limit, tổng số bản ghi và tổng số trang.

4.5.2 Tìm kiếm tin nhắn

Hệ thống hỗ trợ tìm kiếm tin nhắn theo nhiều phạm vi khác nhau, bao gồm toàn bộ cuộc trò chuyện, chỉ các cuộc trò chuyện riêng tư hoặc chỉ các nhóm.

Trước khi thực hiện tìm kiếm, server xây dựng danh sách các cuộc trò chuyện hợp lệ mà người dùng có quyền truy cập. Sau đó, hệ thống áp dụng cơ chế tìm kiếm theo mẫu trên nội dung tin nhắn và tên tệp đính kèm.

Để tối ưu kết quả và tránh trùng lặp, mỗi cuộc trò chuyện chỉ trả về một kết quả tiêu biểu. Đồng thời, số lượng kết quả được giới hạn nhằm đảm bảo hiệu năng và tránh quá tải hệ thống.

4.5.3 Upload file

API POST `/api/messages/upload` sử dụng thư viện Multer để xử lý multipart/form-data. Server phân loại file dựa trên mimetype: image, video, hoặc file thông thường. Đường dẫn file được lưu dưới dạng `/uploads/{filename}` và trả về cùng với meta-data (tên file gốc, kích thước, mimetype) để client đính kèm vào tin nhắn.

4.6 Đếm tin nhắn chưa đọc

Số lượng tin nhắn chưa đọc được tính toán thông qua MongoDB Aggregation Pipeline thay vì load từng tin nhắn riêng lẻ, đảm bảo hiệu năng kể cả khi có nhiều cuộc trò chuyện.

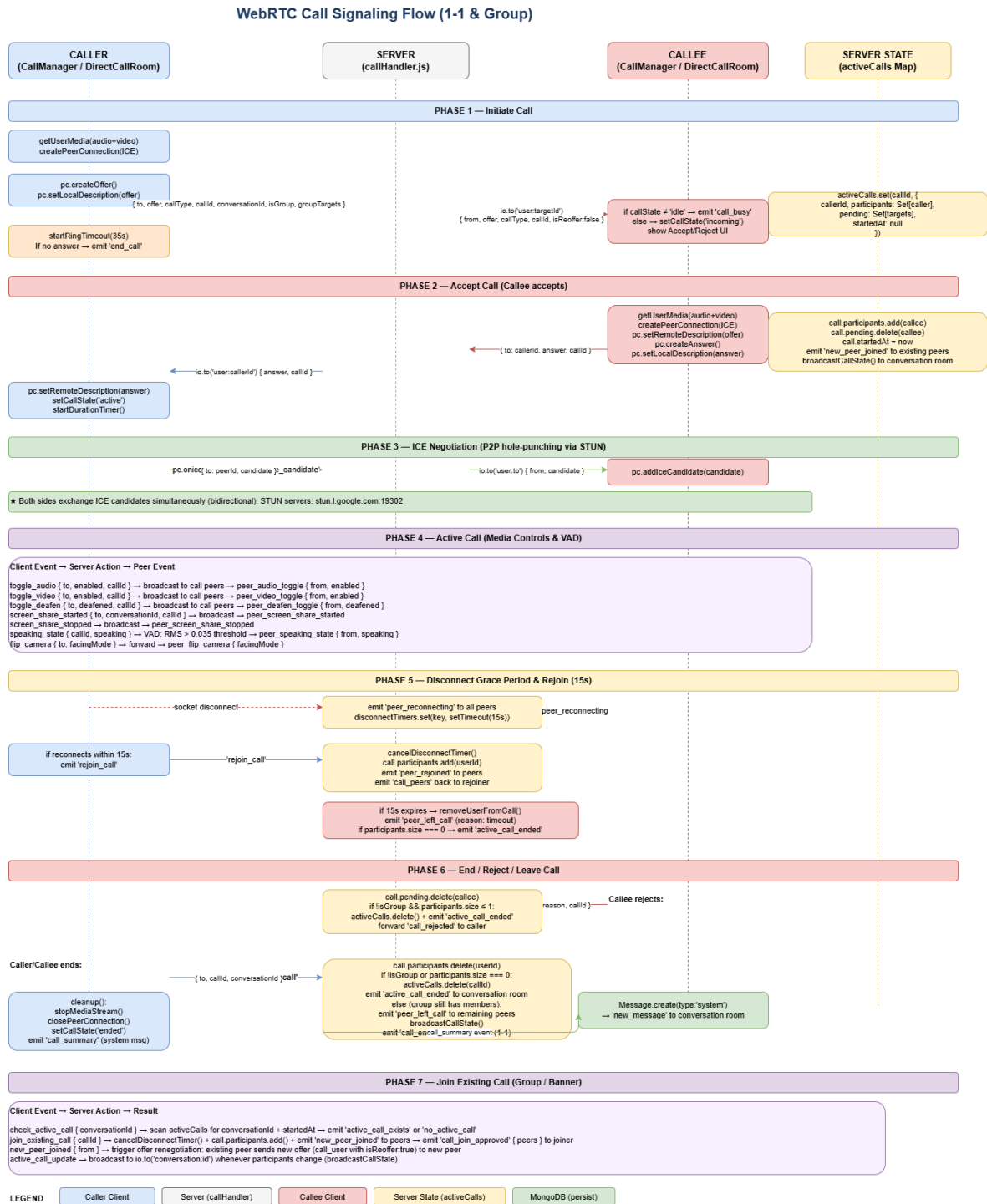
Kết quả được ánh xạ thành unreadMap (dictionary theo conversationId) và gắn vào từng conversation trước khi trả về client. Phương pháp này chỉ cần một lần round-trip tới database thay vì N lần ($N = \text{số conversation}$).

CHƯƠNG 5 - TÍCH HỢP GỌI ĐIỆN WEBRTC

Chương này trình bày quá trình tích hợp chức năng gọi điện Audio/Video thời gian thực vào hệ thống, dựa trên công nghệ WebRTC (Web Real-Time Communication). Nội dung bao gồm kiến trúc tổng thể, cơ chế signaling qua Socket.io, quy trình thiết lập kết nối Peer-to-Peer, và các tính năng điều khiển cuộc gọi nâng cao như Voice Activity Detection (VAD), chia sẻ màn hình và xử lý mất kết nối.

5.1 Tổng quan WebRTC và vai trò của Signaling Server

WebRTC là chuẩn mở cho phép hai trình duyệt truyền tải dữ liệu âm thanh và hình ảnh trực tiếp với nhau (Peer-to-Peer) mà không cần server trung gian xử lý media stream. Tuy nhiên, để thiết lập được kết nối P2P, hai peer cần trao đổi thông tin về cách kết nối (địa chỉ IP, cổng, codec được hỗ trợ) – quá trình này gọi là *signaling*. Hệ thống sử dụng Socket.io làm signaling channel, đóng vai trò trung gian chuyển tiếp các bản tin SDP (Session Description Protocol) và ICE candidates giữa hai trình duyệt.



Hình 5.1 Sơ đồ luồng thiết lập cuộc gọi WebRTC qua Socket.io

Cấu hình ICE server được định nghĩa nhất quán trong ba vị trí: `callConfig.js` (cấu hình frontend), `webRTCHelper.js` (helper functions), và `useWebRTC.js` (hook). Hệ thống sử dụng hai STUN server của Google để hỗ trợ NAT traversal.

STUN server giúp peer khám phá địa chỉ IP công khai của mình. Trong môi trường production, TURN server có thể được thêm vào để xử lý các trường hợp mạng có firewall nghiêm ngặt mà STUN không vượt qua được.

5.2 Kiến trúc hệ thống gọi điện phía server

5.2.1 Quản lý trạng thái cuộc gọi trong bộ nhớ

Module `callHandler.js` quản lý toàn bộ trạng thái cuộc gọi thông qua hai cấu trúc dữ liệu trong bộ nhớ server:

```
const activeCalls = new Map(); // callId → call state object
const disconnectTimers = new Map(); // '${callId}:${userId}' → timerId
```

Mỗi bản ghi trong `activeCalls` lưu trữ đối tượng trạng thái cuộc gọi với các thông tin: `callerId`, `callType` (audio/video), `participants` (Set chứa `userId` đang trong phòng), `pending` (Set chứa `userId` đang chờ chuông chưa trả lời), `startedAt` (null cho đến khi có người accept), và `isGroup` (phân biệt gọi nhóm và 1-1).

Việc sử dụng Set cho `participants` và `pending` đảm bảo không có `userId` trùng lặp và hỗ trợ kiểm tra tư cách thành viên với độ phức tạp $O(1)$.

5.2.2 Thiết kế Signaling Server

Server đóng vai trò relay thuần túy (pure relay) cho các bản tin WebRTC – không can thiệp vào nội dung SDP hay ICE candidate, chỉ chuyển tiếp đến đúng người nhận dựa trên `userId`. Bảng dưới tổng hợp các sự kiện socket được xử lý phía server:

Sự kiện (Client → Server)	Hành động Server	Sự kiện phát ra
call_user	Tạo call entry, ghi pending	incoming_call → callee
accept_call	Thêm vào participants, bắt startedAt	call_accepted → caller; new_peer_joined → peers
reject_call	Xóa khỏi pending	call_rejected → caller
leave_call	Xóa khỏi participants	peer_left_call → còn lại
end_call	Xóa call entry (nếu cuối)	active_call_ended → conversation
ice_candidate	Forward candidate	ice_candidate → peer
rejoin_call	Hủy grace timer, add lại	peer_rejoined → peers; call_peers → user
join_existing_call	Thêm vào participants	new_peer_joined → peers; call_join_approved → user
check_active_call	Tra cứu activeCalls	active_call_exists hoặc no_active_call
toggle_video	Forward trạng thái	peer_video_toggle → peer
toggle_audio	Forward trạng thái	peer_audio_toggle → peer
speaking_state	Forward VAD state	peer_speaking_state → peers
call_summary	Tạo system message	new_message → conversation

Bảng 5.1 Bảng sự kiện Socket.io trong hệ thống gọi điện WebRTC

5.2.3 Cơ chế Grace Period khi mất kết nối

Khi socket của một user trong cuộc gọi bị ngắt đột ngột (sự kiện disconnect), server không xóa user khỏi cuộc gọi ngay lập tức mà áp dụng cơ chế grace period 15 giây (DISCONNECT_GRACE_MS = 15000):

1. Server gửi sự kiện peer_reconnecting tới các người dùng còn lại để giao diện hiển thị trạng thái “đang kết nối lại”.

2. Hệ thống thiết lập một bộ đếm thời gian 15 giây để theo dõi trạng thái kết nối lại của người dùng.
3. Nếu người dùng kết nối lại trong khoảng thời gian này và gửi sự kiện `rejoin_call`, bộ đếm sẽ bị hủy. Người dùng được thêm lại vào danh sách tham gia và các thành viên khác nhận được sự kiện `peer_rejoined` để thiết lập lại kết nối WebRTC.
4. Nếu sau 15 giây người dùng không kết nối lại, hệ thống sẽ tự động loại khỏi cuộc gọi và gửi sự kiện `peer_left_call` tới các thành viên còn lại, kèm theo lý do là hết thời gian chờ.

Cơ chế này đặc biệt hữu ích trên môi trường mobile khi mạng không ổn định, tránh việc cuộc gọi bị gián đoạn không cần thiết do mất kết nối tạm thời.

5.3 Luồng thiết lập cuộc gọi 1-1

5.3.1 *Phía người gọi (Caller)*

Khi người dùng bấm nút gọi, hàm `startCall(type)` trong `CallManager.jsx` thực hiện tuần tự các bước sau:

1. Cập nhật trạng thái UI sang `outgoing`, gọi `getLocalStream(type)` để thu thập luồng âm thanh và hình ảnh từ thiết bị người dùng.
2. Gọi `createPC(otherUser._id)` để khởi tạo `RTCPeerConnection` với cấu hình ICE server. Các track từ local stream được gắn vào PC bằng `addTrack()`.
3. Tạo SDP offer bằng `pc.createOffer()`, đặt làm local description, sau đó phát sự kiện `call_user` kèm offer tới server.
4. Khởi động bộ đếm thời gian đồ chuông 35 giây (`startRingTimeout`). Nếu hết thời gian mà chưa có phản hồi, hệ thống tự động hủy cuộc gọi và ghi nhận trạng thái `missed`.

5.3.2 *Phía người nhận (Callee)*

Khi nhận sự kiện `incoming_call` từ server, giao diện chuyển sang màn hình đổ chuông. Nếu người dùng nhấn chấp nhận, hàm `acceptCall()` thực hiện:

1. Khi phát hiện mất kết nối, server gửi sự kiện `peer_reconnecting` tới các thành viên còn lại để hiển thị trạng thái “đang kết nối lại”.
2. Hệ thống thiết lập một khoảng thời gian chờ 15 giây để người dùng có thể kết nối lại.
3. Nếu người dùng kết nối lại trong thời gian này và gửi sự kiện `rejoin_call`, hệ thống hủy trạng thái chờ, thêm người dùng trở lại cuộc gọi và gửi sự kiện `peer_rejoined` tới các thành viên khác để tái thiết lập kết nối.
4. Nếu quá thời gian chờ mà không có kết nối lại, người dùng sẽ bị loại khỏi cuộc gọi và hệ thống gửi sự kiện `peer_left_call` tới các thành viên còn lại, kèm theo lý do hết thời gian chờ.

5.3.3 *Trao đổi ICE Candidate*

Sau khi SDP được trao đổi, mỗi `RTCPeerConnection` tự động thu thập các ICE candidate (địa chỉ kết nối có thể). Callback `onicecandidate` gửi từng candidate qua socket.

Server nhận và chuyển tiếp candidate tới peer. Khi peer nhận `ice_candidate`, candidate được thêm vào PC bằng `pc.addIceCandidate()`. Hàm `addIceCandidate()` trong `webrtcHelper.js` có bảo vệ thêm: chỉ thêm candidate khi `remoteDescription` đã được đặt (`pc.remoteDescription?.type` phải tồn tại), tránh lỗi `InvalidStateError`.

5.3.4 *Nhận luồng media từ peer*

Khi kết nối P2P được thiết lập thành công, callback `pc.ontrack` nhận `MediaStream` từ peer, sau đó video từ peer được gắn trực tiếp vào phần tử `<video>` thông qua

srcObject – browser tự giải mã và hiển thị stream mà không qua server trung gian. Đây là đặc trưng cốt lõi của WebRTC: media đi P2P, signaling đi qua server.

5.4 Kiến trúc frontend WebRTC

5.4.1 Phân tách trách nhiệm giữa các module

Hệ thống WebRTC phía client được tổ chức thành ba lớp tách biệt rõ ràng:

Module	Trách nhiệm
webrtcHelper.js	Hàm tiện ích low-level: createPeerConnection(), createOffer(), createAnswer(), setLocalDescription(), setRemoteDescription(), addICECandidate(), getUserMedia(), stopMediaStream(), getPeerConnectionStats()
useWebRTC.js (hook)	Logic WebRTC trạng thái cao hơn: quản lý Map các PeerConnection, VAD (Voice Activity Detection), sync local preview, screen share, cleanup toàn bộ
CallManager.jsx	Component UI: quản lý trạng thái cuộc gọi, xử lý sự kiện socket, render các màn hình (incoming/outgoing/active/ended/rejected)
callConfig.js	Hằng số cấu hình: timeout, ICE servers, media constraints, VAD threshold

Bảng 5.2 Phân tách trách nhiệm giữa các module WebRTC phía client

5.4.2 Custom Hook useWebRTC

Hook useWebRTC là trái tim của logic WebRTC phía client, đóng gói toàn bộ sự phức tạp để các component UI chỉ cần gọi các hàm bậc cao. Hook quản lý hai Map quan trọng thông qua useRef: pcMapRef (Map từ peerId tới RTCPeerConnection) và audioAnalysisRef (Map từ userId tới hàm dừng VAD).

Hàm createPC(targetId) là trọng tâm của hook, với điểm thiết kế quan trọng: sử dụng addTrack() trực tiếp thay vì addTransceiver() rỗng. Lý do kỹ thuật:

- `addTrack(track, stream)`: track được đưa vào SDP ngay từ `createOffer()` đầu tiên, peer nhận được media ngay khi kết nối.
- `addTransceiver()` rộng kết hợp `replaceTrack()` sau: có thể xảy ra race condition – `replaceTrack()` chạy trước khi `setRemoteDescription()` hoàn thành, dẫn đến peer không nhận được video.

Hàm `syncPC(pc, forcedVideoTrack)` xử lý trường hợp cần cập nhật track đang chạy (bật/tắt camera, chuyển sang screen share): nếu PC đã có sender thì dùng `replaceTrack()`, nếu chưa thì `addTrack()` mới.

5.4.3 Quản lý trạng thái cuộc gọi trong *CallManager*

`CallManager.jsx` quản lý vòng đời cuộc gọi thông qua state machine với các trạng thái: `idle` → `outgoing/incoming` → `active` → `ended/rejected` → `idle`. Các chuyển trạng thái được điều khiển bởi sự kiện socket và thao tác của người dùng.

Khi kết nối WebRTC chuyển sang trạng thái không còn hoạt động (như `disconnected`, `failed` hoặc `closed`), hệ thống sẽ tự động thực hiện quá trình dọn dẹp tài nguyên. Đồng thời, trạng thái cuối của cuộc gọi được xác định (ví dụ: kết thúc, bị hủy hoặc cuộc gọi nhỡ), sau đó khởi động bộ đếm thời gian 5 giây trước khi đưa hệ thống về trạng thái chờ.

5.5 Các tính năng nâng cao trong cuộc gọi

5.5.1 Voice Activity Detection (VAD)

Hệ thống triển khai VAD (phát hiện giọng nói) bằng Web Audio API để hiển thị chỉ báo trực quan khi ai đó đang nói. VAD hoạt động bằng cách phân tích biên độ âm thanh theo thời gian thực thông qua `AnalyserNode`:

1. Tạo `AudioContext` và `AnalyserNode` (`fftSize = 256`).
2. Kết nối `MediaStream` vào `analyser` qua `createMediaStreamSource()`.

3. Trong mỗi animation frame, đọc dữ liệu biên độ (`getByteTimeDomainData`), tính RMS (Root Mean Square).
4. So sánh RMS với ngưỡng 0.035: nếu vượt ngưỡng, đánh dấu `lastActive = now`.
5. Trạng thái `speaking = (now - lastActive < 550ms)`: nếu 550ms trôi qua mà không có âm thanh vượt ngưỡng, chuyển sang trạng thái im lặng.
6. Khi trạng thái thay đổi, phát sự kiện `speaking_state` qua socket tới các peer trong cuộc gọi.

Để tránh spam socket, hệ thống trong `CallManager` sử dụng `EMIT_COOLDOWN = 150ms` – chỉ emit khi trạng thái thay đổi hoặc sau mỗi 1.5 giây nếu đang nói liên tục (trong hook `useWebRTC`). Khi mic bị tắt, VAD tự ngừng emit và buộc trạng thái `speaking` về `false`.

5.5.2 *Điều khiển media*

Hàm `toggleMic()` tắt/bật track audio bằng `track.enabled = next` thay vì xóa track. Cách tiếp cận này giữ nguyên kết nối PC – track vẫn tồn tại nhưng gửi tín hiệu im lặng khi bị tắt. Sau khi cập nhật track, hàm emit `toggle_audio` tới peer và nếu tắt mic thì buộc emit `speaking_state = false` để peer UI cập nhật đúng.

Chức năng chuyển đổi camera trên thiết bị di động cho phép người dùng thay đổi giữa camera trước và sau. Khi thực hiện, hệ thống sẽ dừng luồng video hiện tại, sau đó khởi tạo luồng mới với chế độ camera tương ứng. Luồng video mới được cập nhật trực tiếp vào kết nối hiện có, giúp duy trì cuộc gọi mà không cần thiết lập lại kết nối.

5.5.3 *Chia sẻ màn hình (Screen Share)*

Hook `useWebRTC` cung cấp hai hàm `startScreenShare()` và `stopScreenShare()`. Khi bắt đầu chia sẻ màn hình:

1. Gọi `navigator.mediaDevices.getDisplayMedia()` để mở dialog chọn màn hình.
2. Lưu stream vào `screenStreamRef`.
3. Gọi `syncAllPCs(screenTrack)` để `replaceTrack()` video track của tất cả `PeerConnection` bằng screen track – các peer nhận được màn hình ngay mà không cần renegotiation.

Khi dừng chia sẻ, gọi `syncAllPCs()` không có tham số – hàm `syncPC()` tự động phát hiện và ưu tiên dùng camera track.

5.5.4 Tóm tắt cuộc gọi và thông báo hệ thống

Sau khi cuộc gọi kết thúc, client phát sự kiện `call_summary` tới server kèm theo `callType`, `status` (`ended/cancelled/missed/rejected`) và `duration`. Server tạo tin nhắn system trong conversation với nội dung mô tả cuộc gọi:

- Cuộc gọi thành công: “ *Cuộc gọi video · X phút Y giây*”
- Cuộc gọi nhờ: “ *Cuộc gọi thoại nhờ*”
- Bị từ chối: “ *Cuộc gọi video bị từ chối*”

Tin nhắn này được phát broadcast vào phòng conversation qua `new_message`, hiển thị trong lịch sử chat như bất kỳ tin nhắn nào khác.

5.6 Luồng gọi nhóm

Hệ thống hỗ trợ gọi nhóm thông qua cơ chế mesh P2P: mỗi participant thiết lập kết nối trực tiếp với từng participant khác. Khi có N người trong cuộc gọi, tổng số `PeerConnection` là $N \times (N - 1) / 2$.

Khi người gọi khởi tạo cuộc gọi nhóm, tham số `isGroup = true` và `groupTargets` (mảng `userId` cần mời) được gửi qua `call_user`. Server lặp qua `groupTargets` và phát `incoming_call` tới từng người, thêm họ vào `call.pending`.

Khi một người trong nhóm chấp nhận (`accept_call`), server xác định danh sách `existingParticipants` đang có trong phòng và phát `new_peer_joined` tới từng người trong danh sách đó. Mỗi người hiện tại nhận được thông báo có peer mới và tự thiết lập `PeerConnection` mới với peer đó – tạo nên kết nối full-mesh.

Thành viên trong conversation có thể tham gia cuộc gọi đang diễn ra bất kỳ lúc nào thông qua banner thông báo. Banner hiển thị khi nhận sự kiện `active_call_update` hoặc `active_call_exists`. Khi tham gia, client gửi `join_existing_call`, server xác nhận bằng `call_join_approved` kèm danh sách peers, client thiết lập `PeerConnection` với từng người đang có mặt.

5.7 Xử lý lỗi và độ bền của cuộc gọi

5.7.1 Các trường hợp lỗi media

Hàm `getUserMedia()` trong `webrtcHelper.js` ánh xạ các lỗi trình duyệt sang thông báo tiếng Việt thân thiện với người dùng:

Loại lỗi	Thông báo hiển thị
<code>NotAllowedError</code>	Bạn từ chối cấp quyền truy cập camera/microphone
<code>NotFoundError</code>	Không tìm thấy camera hoặc microphone
<code>NotReadableError</code>	Camera/microphone đang được sử dụng bởi ứng dụng khác
<code>SecurityError</code>	Không thể truy cập do hạn chế bảo mật (HTTPS required)
<code>OverconstrainedError</code>	Thiết bị không đáp ứng yêu cầu cấu hình

Bảng 5.3 Ánh xạ lỗi media WebRTC sang thông báo người dùng

5.7.2 Xử lý cuộc gọi bận

Khi người dùng đang trong cuộc gọi và nhận được cuộc gọi mới, `CallManager` phát hiện trạng thái `callState !== 'idle'` và tự động emit `call_busy` tới người gọi. Phía gọi nhận sự kiện `call_busy` và thông báo cho người dùng rằng người kia

đang bận.

5.7.3 *Thống kê kết nối*

Hàm `getPeerConnectionStats(pc)` trong `webrtcHelper.js` thu thập thống kê WebRTC thông qua `pc.getStats()`: `bytesReceived/bytesSent`, `packetsLost`, `jitter` cho audio và video, kết hợp với trạng thái `connectionState` và `iceConnectionState`. Dữ liệu này có thể được dùng để hiển thị chất lượng mạng hoặc debug.

5.7.4 *Cơ chế lưu tạm ICE Candidate*

Hệ thống kiểm tra trạng thái remote description trước khi thêm ICE Candidate vào kết nối. Nếu remote description chưa được thiết lập, candidate sẽ được lưu tạm và xử lý lại sau khi hoàn tất `setRemoteDescription()`.

Cách tiếp cận này giúp tránh lỗi do thứ tự bất đồng bộ trong signaling, điển hình là lỗi `InvalidStateError`.

5.8 *Luồng Socket.io tổng thể của hệ thống*

Ngoài gọi điện, Socket.io còn là xương sống của toàn bộ tính năng thời gian thực trong hệ thống. Sơ đồ dưới đây minh họa toàn bộ luồng socket từ kết nối đến các handler chuyên biệt:

```

sequenceDiagram
    participant ClientA as Client A (Browser / App)
    participant Server as Socket.io Server (index.js)
    participant Redis as Redis Adapter (User online/offline)
    participant MongoDB as MongoDB (Message / Conversation)
    participant ClientB as Client B (Other participant)

    ClientA->>Server: JWT verify + User.findById(1)
    Note over Server: goOnline() → User.update(isOnline:true) (redis)
    Server->>MongoDB: Auto join user room socket.join('user.userId')
    Note over Server: Verify participant → socket.join('conversation.id')
    Server->>ClientB: { conversationId }ersion'
    Note over Server: Notify friends/participants: user_online'
    Server->>MongoDB: Message.create() + Conversation.update(lastMessage)
    Note over Server: Check: isActiveParticipant() + dissolvedAt?
    ClientA->>Server: { conversationId, content, type, fileUri, replyTo }
    Note over Server: io.to('conversation.id').emit('new_message')
    Note over Server: io.to(user.participantId).emit('new_message')
    Note over Server: Deliver to open chat
    Note over Server: Sidebar unread update
    ClientA->>Server: 'typing_start'
    Note over Server: 'user_typing' / 'user_stop_typing'
    Note over Server: socket.to('conversation.id').emit('user_typing')
    Note over Server: 'new_message'
    Note over Server: 'user_offline'

    %% Internal Decisions
    Note over Server: react_message { messageId, emoji } → update reactions Map → io.to(room).emit('message_reaction')
    Note over Server: edit_message { messageId, content } → push editHistory → io.to(room).emit('message_edited')
    Note over Server: delete_message { messageId } → Message.findByIdAndDelete → io.to(room).emit('message_deleted')
    Note over Server: pin_message { messageId, isPinning } → update pinnedMessages (max 3) → io.to(room).emit('pin_updated')
    Note over Server: message_read { conversationId } → Message.updateMany(readBy) → socket.to(room).emit('messages_read')
  
```

LEGEND

- Client → Server (Blue arrow)
- Server → Client (Red arrow)
- Server → DB (Green arrow)
- Internal Decision (Yellow box)

Khi socket kết nối, middleware xác thực JWT và truy vấn MongoDB để lấy đầy đủ thông tin user (username, email, avatar). Sau đó socket join phòng cá nhân user : {userId} và ba bộ handler được khởi tạo song song: setupPresenceHandlers (quản lý online/offline), setupMessageHandlers (nhắn tin), và setupCallHandlers (gọi điện). Nhờ Redis Adapter, các sự kiện socket được đồng bộ xuyên suốt giữa nhiều

instance backend – người dùng kết nối vào server A vẫn nhận được tin nhắn từ người dùng kết nối vào server B.

CHƯƠNG 6 - GIẢI PHÁP TỐI ƯU HIỆU NĂNG, MỞ RỘNG HỆ THỐNG VÀ BẢO MẬT TOÀN DIỆN

Trong các hệ thống chat real-time hiện đại, ba yếu tố hiệu năng (performance), khả năng mở rộng (scalability) và bảo mật (security) không tồn tại độc lập mà có mối quan hệ ràng buộc lẫn nhau theo các định lý giới hạn của hệ thống phân tán (CAP theorem, Brewer, 2000). Việc tối ưu một yếu tố thường kéo theo sự suy giảm hoặc gia tăng độ phức tạp của các yếu tố còn lại, tạo ra các bài toán đánh đổi (trade-off) mang tính kiến trúc.

Về mặt ràng buộc kiến trúc: một hệ thống hướng đến hiệu năng cao (low-latency) thường phải hy sinh tính nhất quán mạnh (strong consistency) để đạt được tính sẵn sàng (availability); ngược lại, hệ thống tập trung quá mức vào bảo mật đa lớp có thể tạo ra độ trễ xử lý, ảnh hưởng đến trải nghiệm real-time.

Do đó, hệ thống được thiết kế theo hướng kiến trúc phân tán (distributed architecture), kết hợp các nguyên lý stateless backend, caching tầng trung gian, message broker và load balancing nhằm cân bằng giữa ba yếu tố trên. Chương này trình bày theo hai phần: các giải pháp đã triển khai trong hệ thống hiện tại và các đề xuất nâng cấp hướng tới môi trường production.

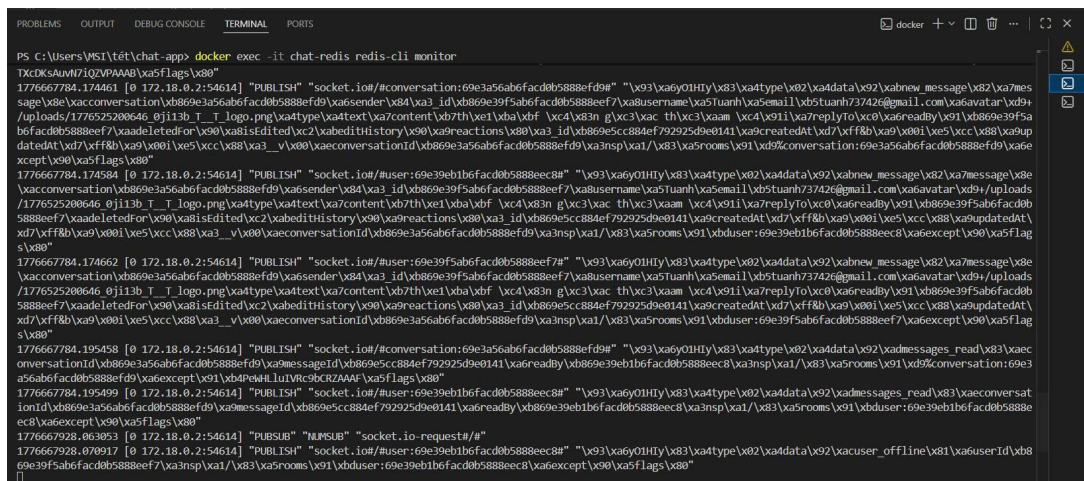
6.1 Các giải pháp đã triển khai trong hệ thống

6.1.1 *Redis – Caching và Pub/Sub phân tán*

Redis được lựa chọn thay vì các giải pháp in-memory khác như Memcached nhờ khả năng hỗ trợ đồng thời cả caching (cấu trúc dữ liệu đa dạng) và Pub/Sub (message broker). Việc chuyển trạng thái từ MongoDB sang Redis giúp giảm tải I/O cho cơ sở dữ liệu chính – vốn là nút cổ chai (bottleneck) trong các mô hình tương tác real-time.

Redis được sử dụng với hai vai trò chính trong hệ thống:

- **Caching layer (Cache-aside pattern):** Thông tin người dùng được cache với TTL 3600 giây. Middleware protect ưu tiên đọc từ Redis trước, chỉ truy vấn MongoDB khi cache miss, sau đó lưu kết quả vào cache. Cơ chế này giảm đáng kể số lượng truy vấn tới MongoDB trong các request xác thực liên tiếp.
- **Message broker (Pub/Sub) qua Socket.io Redis Adapter:** Khi một back-end instance phát sự kiện socket (ví dụ `new_message`), Redis Adapter tự động PUBLISH thông điệp lên kênh tương ứng. Các instance khác đang SUBSCRIBE kênh đó sẽ nhận và forward tới client của mình – đảm bảo tin nhắn được phân phối chính xác dù người gửi và người nhận kết nối vào các server khác nhau.



Hình 6.1 Redis monitor ghi nhận các lệnh PUBLISH khi người dùng gửi tin nhắn – minh chứng Redis Pub/Sub đồng bộ sơ kiến Socket.io theo thời gian thực

Đánh giá trade-off: Đây là sự đánh đổi giữa tính nhất quán (Consistency) và

tính sẵn sàng (Availability). Cache giúp tăng throughput nhưng tiềm ẩn nguy cơ stale data. Hệ thống giải quyết bằng cơ chế cache invalidation chủ động: mỗi khi profile người dùng thay đổi, hàm `cacheDel()` được gọi ngay lập tức để xóa bản ghi cũ.

Hạn chế hiện tại: Dữ liệu truyền qua Redis Pub/Sub được serialize bằng MessagePack (nhị phân) nhằm giảm băng thông, nhưng chưa được mã hóa. Trong môi trường production, cần bổ sung Redis AUTH password và TLS để bảo vệ dữ liệu trong kênh truyền nội bộ.

6.1.2 Kiến trúc Stateless và Horizontal Scaling

Để đạt được khả năng mở rộng theo chiều ngang (horizontal scalability), backend được thiết kế hoàn toàn stateless – toàn bộ trạng thái (socket mapping, session, cache người dùng) được tách riêng và quản lý tập trung tại Redis. Mô hình này cho phép thêm instance backend mới bất kỳ lúc nào mà không cần thay đổi cấu hình hệ thống, chỉ cần chạy lệnh:

```
docker compose up -d --scale backend=2
```

Socket.io Redis Adapter đảm bảo sự kiện được đồng bộ xuyên suốt: user kết nối vào instance A vẫn nhận được tin nhắn từ user kết nối vào instance B thông qua kênh Pub/Sub dùng chung.

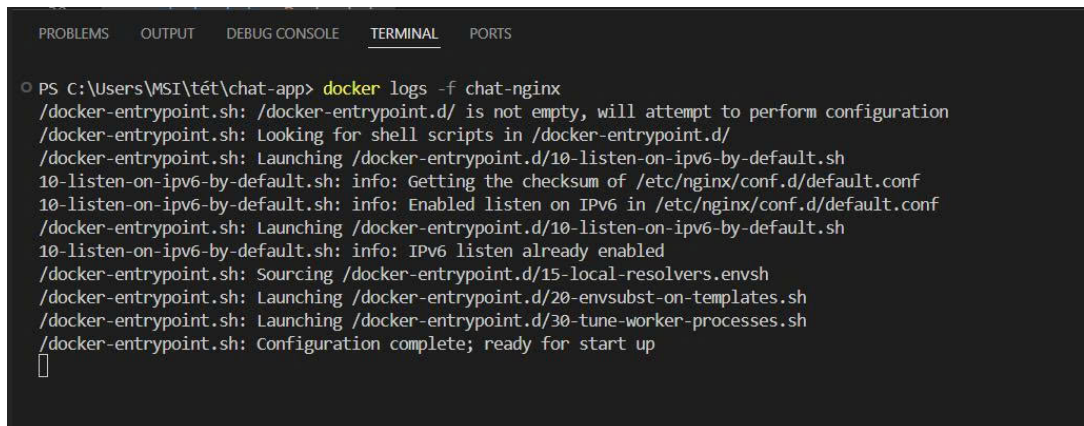
Đánh giá trade-off: Mô hình stateless mang lại khả năng mở rộng gần tuyến tính (linear scalability) nhưng đồng thời gia tăng độ trễ mạng do mỗi request phải tương tác với Redis, tạo ra điểm phụ thuộc (single point of dependency) cần được giám sát chặt chẽ.

6.1.3 Nginx – Reverse Proxy và Load Balancing

Nginx được lựa chọn làm entry gateway vì hai lý do chính: xử lý static content và SSL termination hiệu quả hơn HAProxy, đồng thời sử dụng kiến trúc event-driven giúp tiêu thụ ít RAM hơn Apache khi duy trì hàng nghìn kết nối WebSocket dài (long-

lived connections).

Chiến lược `ip_hash` đảm bảo mỗi client luôn được định tuyến tới cùng một backend instance, giữ ổn định kết nối WebSocket và giảm chi phí đồng bộ qua Redis Pub/Sub.



```
PS C:\Users\MSI\tét\chat-app> docker logs -f chat-nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: IPv6 listen already enabled
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
```

Hình 6.2 Log Nginx ghi nhận các request được proxy tới backend – minh chứng Nginx hoạt động như reverse proxy phân phối traffic

Đánh giá trade-off: Sticky session giảm tải cho Redis Pub/Sub nhưng có thể gây mất cân bằng tải (skewed load distribution) trong môi trường NAT khi nhiều client chung một IP công khai. Đây là chấp nhận được trong bối cảnh chat, nơi độ ổn định WebSocket được ưu tiên hơn cân bằng tải tuyệt đối.

6.1.4 Tối ưu MongoDB

Chiến lược indexing được thiết kế dựa trên phân tích mẫu truy vấn thực tế. Các truy vấn chậm nhất trong hệ thống chat thường rơi vào phân trang lịch sử tin nhắn và tìm kiếm hội thoại:

- **Compound Index** (`conversation, createdAt: -1`): Hỗ trợ truy vấn phân trang `find({conversation}).sort({createdAt: -1}).limit(50)` với hiệu năng cao.
- **Index participants**: Tối ưu truy vấn tìm kiếm hội thoại theo thành viên.

- **Unique Index email:** Đảm bảo ràng buộc toàn vẹn dữ liệu ở cấp database.

Ngoài ra, **Projection** được áp dụng trong các truy vấn populate để chỉ lấy các trường cần thiết (ví dụ `select('username avatar email')`), giảm băng thông giữa tầng ứng dụng và database.

6.1.5 Cơ chế xác thực và bảo vệ cơ bản

Hệ thống áp dụng JWT (HS256) theo cơ chế xác thực không trạng thái (stateless authentication), phù hợp với kiến trúc phân tán. Token được xác thực qua header `Authorization: Bearer`, và Socket.io handshake được bảo vệ bằng cách xác thực token ngay từ giai đoạn kết nối – trước khi bất kỳ sự kiện nào được xử lý. CORS được cấu hình theo origin whitelist như lớp bảo vệ cơ bản chống truy cập trái phép từ tên miền khác.

```

PS C:\Users\MSI\tet\chat-app> docker logs -f chat-backend
GET /api/friends/requests 304 16.666 ms - -
Cache set error: Error: Redis client not initialized
  at getRedisClient (file:///app/src/config/redis.js:29:11)
  at cacheSet (file:///app/src/config/redis.js:37:20)
  at protect (file:///app/src/middlewares/auth.middleware.js:35:15)
  at process.processTicksAndRejections (node:internal/process/task_queues:95:5)
✅ User connected: Tu Anh (69e39eb1b6facd0b5888eec8) Socket: PeWHLluIVRc9bCRZAAAF
Cache get error: Error: Redis client not initialized
  at getRedisClient (file:///app/src/config/redis.js:29:11)
  at cacheGet (file:///app/src/config/redis.js:48:20)
  at protect (file:///app/src/middlewares/auth.middleware.js:23:24)
  at Layer.handle [as handle_request] (/app/node_modules/express/lib/router/layer.js:95:5)
  at trim prefix (/app/node_modules/express/lib/router/index.js:328:13)
  at /app/node_modules/express/lib/router/index.js:286:9
  at Function.process_params (/app/node_modules/express/lib/router/index.js:346:12)
  at next (/app/node_modules/express/lib/router/index.js:280:10)
  at Function.handle (/app/node_modules/express/lib/router/index.js:175:3)
  at router (/app/node_modules/express/lib/router/index.js:47:12)
✅ User 69e39eb1b6facd0b5888eec8 is now online
Cache set error: Error: Redis client not initialized
  at getRedisClient (file:///app/src/config/redis.js:29:11)
  at cacheSet (file:///app/src/config/redis.js:37:20)
  at protect (file:///app/src/middlewares/auth.middleware.js:35:15)
  at process.processTicksAndRejections (node:internal/process/task_queues:95:5)
GET /api/conversations 200 35.760 ms - 3824
GET /api/friends/requests 304 9.456 ms - -
Cache get error: Error: Redis client not initialized
  at getRedisClient (file:///app/src/config/redis.js:29:11)
  at cacheGet (file:///app/src/config/redis.js:48:20)

```

Hình 6.3 Log backend ghi nhận kết nối WebSocket, xác thực JWT và xử lý API request theo thời gian thực

6.2 Các đề xuất nâng cấp cho môi trường production

6.2.1 Xác thực đa yếu tố (MFA/TOTP)

Dựa trên mô hình bảo mật đa lớp (defense-in-depth), nghiên cứu đề xuất bổ sung MFA dựa trên TOTP (RFC 6238). Cơ chế này tách biệt yếu tố “kiến thức” (mật khẩu) và yếu tố “sở hữu” (thiết bị tạo OTP), giúp vô hiệu hóa các cuộc tấn công credential stuffing ngay cả khi mật khẩu bị lộ. Chi phí bổ sung là gia tăng độ trễ xác thực khoảng 2–3 giây và phụ thuộc vào đồng bộ thời gian (NTP).

6.2.2 Kiến trúc Access Token / Refresh Token

Đề xuất chuyển từ JWT đơn lớp (TTL 7 ngày) sang mô hình two-tier token: Access token (TTL 15 phút) lưu trong memory, Refresh token (TTL 7 ngày) lưu trong HTTP-only Cookie. Khi Access token hết hạn, client tự động dùng Refresh token để lấy token mới mà không cần đăng nhập lại.

Bổ sung Redis blacklist (token denylist) để hỗ trợ thu hồi token chủ động khi phát hiện bất thường hoặc người dùng đăng xuất toàn bộ thiết bị – khắc phục điểm yếu cố hữu của JWT là không thể thu hồi trước hạn.

6.2.3 Tăng cường bảo vệ hệ thống

Biện pháp	Mục tiêu
Rate limiting (Leaky Bucket / Token Bucket)	Chống brute-force và DDoS
CSRF protection (Synchronizer Token Pattern)	Bảo vệ request thay đổi trạng thái
CSP (Content Security Policy)	Giảm nguy cơ XSS
Giới hạn WebSocket connections theo IP	Kiểm soát tài nguyên, chống DDoS tầng ứng dụng
Redis AUTH + TLS	Mã hóa dữ liệu trong kênh Pub/Sub nội bộ
Backpressure mechanism	Bảo vệ hệ thống khi quá tải

Bảng 6.1 Các biện pháp tăng cường bảo vệ hệ thống

6.2.4 Giám sát và phát hiện tấn công

Hệ thống giám sát được thiết kế theo mô hình SIEM (Security Information and Event Management) thu nhỏ, bao gồm: logging structured JSON để chuẩn hóa sự kiện bảo mật; ELK Stack để phân tích và cảnh báo tập trung; Fail2ban để chặn IP có hành vi tấn công lặp lại dựa trên log parsing; Snyk và Trivy để quét lỗ hổng ở tầng dependency và container image theo hướng shift-left security.

6.2.5 Quy trình ứng phó sự cố

Quy trình gồm 5 giai đoạn theo chuẩn NIST: Phát hiện (Detection), Cách ly (Containment), Phân tích nguyên nhân gốc (Eradication & Root Cause Analysis), Khôi phục (Recovery), và Đánh giá hậu sự cố (Post-Incident Review).

6.3 Phân tích kiến trúc và trade-off tổng hợp

Giải pháp	Lợi ích	Hạn chế / Chi phí
Redis caching	Tăng throughput, giảm I/O DB	Nguy cơ stale data (eventual consistency)
Redis Pub/Sub	Đồng bộ sự kiện giữa các instance	Dữ liệu chưa mã hóa, cần Redis AUTH + TLS
Stateless backend	Linear scalability	Tăng network latency (phụ thuộc Redis)
Sticky session (Nginx)	Ổn định kết nối WebSocket	Mất cân bằng tải trong môi trường NAT
MFA/TOTP	Tăng bảo mật (defense-in-depth)	Gia tăng độ trễ xác thực, phụ thuộc NTP
Refresh token	Giảm exposure window	Tăng complexity authentication flow
Rate limiting	Chống DDoS, bảo vệ tài nguyên	Có thể ảnh hưởng user hợp lệ (false positive)

Bảng 6.2 Tổng hợp trade-off các giải pháp kiến trúc

Hệ thống được thiết kế dựa trên nguyên tắc cân bằng thực nghiệm giữa hiệu năng – bảo mật – độ phức tạp triển khai, phù hợp với bài toán chat real-time có yêu cầu về độ trễ thấp (dưới 100ms) và tính sẵn sàng cao. Các đề xuất nâng cấp ở phần 6.2 đặt nền móng cho lộ trình phát triển hệ thống lên môi trường production trong giai đoạn tiếp theo.

CHƯƠNG 7 - KẾ HOẠCH TRIỂN KHAI DEMO CUỐI KỲ

7.1 Mục tiêu

Kế hoạch triển khai demo hệ thống chat real-time nhằm cung cấp một nền tảng giao tiếp trực tuyến với các chức năng tương tự các ứng dụng nhắn tin hiện đại như Messenger hoặc Zalo. Mục tiêu của demo không chỉ xây dựng một hệ thống chat hoạt động đúng chức năng, mà còn chứng minh khả năng vận hành ổn định end-to-end với đầy đủ các thành phần: realtime messaging, quản lý người dùng, lưu trữ dữ liệu bền vững, và khả năng mở rộng hệ thống.

Cụ thể, demo tập trung vào các nhóm tính năng chính sau:

- **Chat realtime:** Nhắn tin 1-1, nhắn tin nhóm, typing indicator, trạng thái đã đọc (read receipts)
- **Quản lý xã hội (Social):** Kết bạn, tìm kiếm người dùng, danh sách bạn bè, chấp nhận/từ chối lời mời
- **Đa phương tiện (Multimedia):** Gửi file, hình ảnh, emoji, reaction (emoji phản hồi tin nhắn)
- **Gọi thoại/video:** Gọi audio, gọi video qua WebRTC (signaling qua Socket.io)
- **Cá nhân hóa:** Cài đặt giao diện (Sáng/Tối), đổi mật khẩu, lưu trữ cá nhân
- **Thông báo:** Hiển thị thông báo/badge cho tin nhắn mới và lời mời kết bạn
- **Mở rộng hệ thống (Scaling):** Khả năng scale backend lên nhiều instance mà vẫn đảm bảo realtime nhờ Redis Pub/Sub

7.2 Công cụ sử dụng (Tools Required)

Các công cụ và môi trường sử dụng để xây dựng và triển khai hệ thống bao gồm:

- **Ngôn ngữ lập trình:** JavaScript/TypeScript (Node.js cho backend, React cho frontend)

- **Framework backend:** Express.js (xây dựng REST API và WebSocket server)
- **Framework frontend:** ReactJS (xây dựng giao diện người dùng)
- **Thư viện realtime:** Socket.io (quản lý WebSocket connection)
- **Cơ sở dữ liệu:** MongoDB (lưu trữ users, messages, conversations, groups)
- **Cache và Pub/Sub:** Redis (cache session, đồng bộ realtime giữa các instance)
- **Reverse Proxy:** Nginx (cân bằng tải, proxy WebSocket)
- **Sinh QR Code:** Thư viện qrcode (cho feature xác thực nếu có)
- **Containerization:** Docker + Docker Compose
- **WebRTC:** WebRTC API + Socket.io signaling
- **Môi trường triển khai:** Ubuntu / Windows + WSL / macOS
- **Công cụ kiểm thử:** Trình duyệt (DevTools), curl, Postman

7.3 Kịch bản Demo (Scenario)

Kịch bản demo được xây dựng theo luồng nghiệp vụ thực tế, từ khi người dùng truy cập hệ thống đến khi sử dụng đầy đủ các tính năng. Demo bao gồm 15 kịch bản con, được thực hiện tuần tự trên môi trường đã triển khai bằng Docker Compose với 5 services: frontend, backend, mongodb, redis, nginx.

Chuẩn bị trước demo:

- Hệ thống đã được khởi chạy bằng Docker Compose (5 containers ở trạng thái Up)
- 02 tài khoản demo: userA, userB (có thể đăng ký trực tiếp trong lúc demo)
- 02 trình duyệt/phiên: Chrome (userA) và Chrome ẩn danh (userB)
- Camera/mic đã được cấp quyền (nếu demo WebRTC)
- 01 đường link mẫu (ví dụ: <https://youtube.com/watch?v=abc>) để test nhận diện link

Demo 7.3.0 — Kiểm tra hệ thống (Xác nhận đã chạy)

- Truy cập `http://localhost` (qua Nginx) → Giao diện load thành công
- Đăng nhập vào ứng dụng → Chuyển đến trang chat chính

Kết quả mong đợi: Hệ thống hoạt động bình thường, sẵn sàng demo.

Demo 7.3.1 — Đăng ký và đăng nhập (Xác thực JWT)

- Tạo tài khoản mới (hoặc dùng tài khoản có sẵn) → Đăng ký thành công
- Đăng nhập bằng tài khoản vừa tạo → Nhận JWT token
- Kiểm tra header request API sau login → Có Bearer token

Kết quả mong đợi: JWT hoạt động chính xác, xác thực người dùng ổn định.

(3) Demo 3.3.2 — Tìm kiếm người dùng và kết bạn

- userA tìm kiếm userB theo username/email → Hiển thị đúng userB
- userA gửi lời mời kết bạn → userB nhận thông báo realtime
- userB chấp nhận lời mời → Danh sách bạn bè cập nhật ở cả hai phía

Kết quả mong đợi: Kết bạn và danh sách bạn bè cập nhật realtime.

Demo 7.3.3 — Chat realtime 1-1, trạng thái người dùng và trạng thái đã đọc

- userA mở hội thoại với userB → Hội thoại được tạo
- userA gửi tin nhắn "Hello realtime" → userB nhận ngay lập tức
- Kiểm tra trạng thái online/offline → Hiển thị đúng
- userB bắt đầu gõ → userA thấy "userB is typing..."

- userB mở chat và xem tin nhắn → userA thấy trạng thái đã đọc
- Refresh trình duyệt userB → Lịch sử tin nhắn vẫn còn (MongoDB)

Kết quả mong đợi: Hệ thống chat hoạt động giống ứng dụng thực tế.

Demo 7.3.4 — Upload file, ảnh và các thao tác tin nhắn nâng cao

- userA gửi ảnh/file → userB thấy hiển thị đúng
- userB thả reaction (emoji) → Reaction hiển thị realtime
- userB reply tin nhắn → Hiển thị kèm nội dung gốc
- userA edit tin nhắn → Nội dung thay đổi ở cả hai phía
- userA delete tin nhắn → Tin nhắn biến mất

Kết quả mong đợi: Multimedia và interaction hoạt động ổn định.

Demo 7.3.5 — Tìm kiếm tin nhắn trong lịch sử chat

- Nhập keyword vào thanh tìm kiếm → Hiển thị tin nhắn chứa keyword
- Lọc kết quả theo người gửi hoặc thời gian → Kết quả chính xác

Kết quả mong đợi: Tìm kiếm trong lịch sử chat chính xác.

Demo 7.3.6 — Xử lý link (URL) trong tin nhắn

- userA gửi URL (ví dụ: <https://youtube.com/watch?v=abc>) → Link được nhận diện và hiển thị đúng định dạng (clickable)
- userB xem tin nhắn → Link hiển thị đúng, có thể click để mở

Ghi chú: Nếu hệ thống có hỗ trợ preview (fetch metadata) thì hiển thị thêm khung preview; nếu chưa thì đây là hướng mở rộng cho các phiên bản sau.

Kết quả mong đợi: Link được nhận diện và hiển thị đúng.

Demo 7.3.7 — Emoji và Reaction

- userA gửi emoji trong tin nhắn → Emoji hiển thị đúng
- userB thả reaction vào tin nhắn → Reaction đồng bộ realtime

Kết quả mong đợi: Emoji và reaction đồng bộ realtime.

Demo 7.3.8 — Chat nhóm (Group chat)

- userA tạo group mới → Group được tạo thành công
- userA thêm userB vào group → userB xuất hiện trong danh sách thành viên
- userA gửi tin nhắn trong group → userB nhận realtime

Kết quả mong đợi: Chat nhóm hoạt động realtime.

Demo 7.3.9 — Gọi audio và video (WebRTC)

- userA gọi video (hoặc audio) cho userB → userB nhận thông báo
- userB accept call → Kết nối WebRTC được thiết lập
- Bật/tắt mic, camera → Trạng thái cập nhật đúng
- Kết thúc cuộc gọi → Cả hai kết thúc thành công

Lưu ý: WebRTC phụ thuộc vào quyền truy cập camera/mic của trình duyệt.

Kết quả mong đợi: Cuộc gọi realtime hoạt động ổn định.

Demo 7.3.10 — Thông báo (Notification / Badge)

- userA gửi tin nhắn cho userB → Có hiển thị thông báo/badge
- userA gửi lời mời kết bạn cho userB → Có hiển thị thông báo/badge
- userB mở xem → Trạng thái thông báo được cập nhật

Kết quả mong đợi: Thông báo dạng badge hoạt động đúng.

Demo 7.3.11 — Cài đặt giao diện (Sáng/Tối - Light/Dark mode)

- Vào Settings → Chuyển sang Dark mode → Giao diện chuyển sang tối
- Chuyển sang Light mode → Giao diện chuyển về sáng
- Refresh trang → Giữ đúng theme đã chọn (nếu có lưu)

Kết quả mong đợi: UI thay đổi mượt mà, trạng thái được lưu.

Demo 7.3.12 — Lưu trữ cá nhân (Personal Storage)

- Mở Personal Storage → Hiển thị danh sách các mục
- Tạo mới ghi chú (Create) → Xuất hiện trong danh sách
- Sửa nội dung (Update) → Được cập nhật
- Xóa ghi chú (Delete) → Biến mất
- Refresh trang → Dữ liệu còn lại vẫn hiển thị

Kết quả mong đợi: CRUD hoạt động, dữ liệu bền vững.

Demo 7.3.13 — Đổi mật khẩu (Change Password)

- Vào Profile/Security → Đổi mật khẩu
- Nhập mật khẩu cũ + mật khẩu mới → Xác nhận thành công
- Đăng xuất → Về trang login
- Đăng nhập bằng mật khẩu mới → Thành công
- Đăng nhập bằng mật khẩu cũ → Không thành công

Kết quả mong đợi: Bảo mật hoạt động đúng.

Demo 7.3.14 — Mở rộng hệ thống (Horizontal Scaling với Redis Pub/Sub)

Mục tiêu: Chứng minh hệ thống vẫn đảm bảo realtime khi backend được scale lên nhiều instance.

- Scale backend lên 2-3 instance:

```
docker compose up --scale backend=3 -d
```

- userA và userB kết nối vào hệ thống → Cả hai vẫn gửi/nhận tin nhắn được
- userA gửi tin nhắn cho userB → userB nhận được tin nhắn (Redis Pub/Sub đồng bộ)
- (Tùy chọn) Restart một backend instance:

```
docker restart backend-1
```

- Client tự động reconnect, không mất tin nhắn

Kết quả mong đợi: Redis Pub/Sub đảm bảo đồng bộ realtime khi scale.

7.4 Kế hoạch triển khai (Deployment Plan)

Hệ thống được triển khai bằng Docker Compose theo các bước tuần tự dưới đây.

Bước 1: Chuẩn bị môi trường

- Cài đặt Docker (phiên bản 20.10 trở lên) và Docker Compose (phiên bản 2.0 trở lên)
- Tạo file cấu hình biến môi trường `server/.env` từ mẫu `server/.env.example` với các tham số: `PORT`, `MONGODB_URI`, `REDIS_URL`, `JWT_SECRET`, `CLIENT_URL`
- Tạo các Docker volumes: `mongodb_data`, `redis_data`, `uploads_data`

Bước 2: Xây dựng cấu hình Docker Compose

- Tạo file `docker-compose.yml` định nghĩa 5 services: frontend, backend, mongodb, redis, nginx
- Cấu hình Nginx làm reverse proxy: route `/api/` đến backend, route `/socket.io/` hỗ trợ WebSocket upgrade

Bước 3: Build và khởi chạy toàn bộ hệ thống

- Chạy lệnh `docker compose build`
- Chạy lệnh `docker compose up -d`
- Kiểm tra trạng thái: `docker compose ps` (5 containers "Up")

Bước 4: Kiểm tra sức khỏe hệ thống (Smoke Test)

- Kiểm tra Nginx: Mở trình duyệt `http://localhost` → Hiện thị giao diện chat
- Kiểm tra API: `curl http://localhost/health` (hoặc endpoint health tương đương) → HTTP 200
- Kiểm tra Socket.io: Mở DevTools → Network → WS → Kết nối thành công
- Kiểm tra MongoDB: `docker exec -it chat-mongodb mongosh -eval "db.adminCommand('ping')"`
- Kiểm tra Redis: `docker exec -it chat-redis redis-cli ping` → PONG

Bước 5: Mở rộng hệ thống (Horizontal Scaling) - Tùy chọn

- Scale backend: `docker compose up -d --scale backend=3`
- Đảm bảo Nginx cân bằng tải giữa các instance
- Đảm bảo Redis Pub/Sub đồng bộ sự kiện Socket.io

7.5 Tiêu chí đánh giá thành công (Success Criteria)

Demo được xem là thành công khi đáp ứng các tiêu chí sau:

Tiêu chí kỹ thuật:

- Toàn bộ 5 containers (frontend, backend, mongodb, redis, nginx) chạy ổn định
- API hoạt động qua Nginx (reverse proxy) và trả về đúng dữ liệu
- Socket.io kết nối realtime thành công giữa nhiều client
- Dữ liệu được lưu trữ bền vững trong MongoDB sau refresh
- Redis hoạt động ổn định cho cache và Pub/Sub

Tiêu chí chức năng:

- Đăng ký và đăng nhập thành công, JWT token được cấp và xác thực đúng
- Tìm kiếm người dùng và kết bạn hoạt động đúng luồng
- Nhắn tin realtime 1-1: tin nhắn đến ngay lập tức, typing indicator và trạng thái đã đọc hiển thị đúng
- Gửi được file, ảnh, emoji, reaction trong chat
- Tìm kiếm tin nhắn theo keyword trả về kết quả chính xác
- Link trong tin nhắn được nhận diện và hiển thị đúng (clickable)
- Chat nhóm hoạt động realtime với nhiều thành viên
- Gọi audio/video (WebRTC) thiết lập kết nối thành công
- Có hiển thị thông báo/badge khi có tin nhắn mới hoặc lời mời kết bạn
- Cài đặt giao diện sáng/tối thay đổi được và có thể lưu trạng thái
- Lưu trữ cá nhân (Personal Storage) hỗ trợ đầy đủ CRUD
- Đổi mật khẩu thành công, mật khẩu cũ hết hiệu lực

Tiêu chí mở rộng:

- Backend có thể scale lên 2-3 instance thành công
- Nhắn tin realtime vẫn hoạt động ổn định sau khi scale
- Redis Pub/Sub đảm bảo đồng bộ sự kiện giữa các instance

7.6 Kết quả mong đợi (Expected Results)

Sau khi hoàn thành kế hoạch triển khai demo, hệ thống sẽ đạt được:

- Hệ thống chat real-time hoạt động ổn định end-to-end trên môi trường Docker Compose với 5 services
- Các tính năng realtime (nhắn tin, typing, trạng thái đã đọc, gọi WebRTC) hoạt động chính xác và mượt mà
- Dữ liệu người dùng, tin nhắn, bạn bè, nhóm chat được lưu trữ bền vững trong MongoDB
- Redis hoạt động hiệu quả cho cache và Pub/Sub, đảm bảo realtime khi scale ngang
- Giao diện người dùng thân thiện, hỗ trợ đầy đủ các tương tác cơ bản
- Hệ thống có khả năng mở rộng: backend có thể scale lên nhiều instance mà vẫn đồng bộ realtime
- Minh họa thành công một ứng dụng chat hiện đại với các tính năng tương tự Messenger hoặc Zalo, có thể làm nền tảng để phát triển thêm các tính năng nâng cao trong tương lai (preview link metadata, end-to-end encryption, monitoring, CI/CD)

TÀI LIỆU THAM KHẢO

- Banks, A. and Porcello, E. (2017). *Learning React: Functional Web Development with React and Redux*. O'Reilly Media.
- bcryptjs Contributors (2024). bcryptjs – npm package. [Online].
- Carlson, J. L. (2013). *Redis in Action*. Manning Publications.
- Chodorow, K. (2013). *MongoDB: The Definitive Guide*. O'Reilly Media, 2nd edition.
- Docker Inc. (2024). Docker documentation. [Online].
- Evan You and Vite Contributors (2024). Vite documentation. [Online].
- F5 Networks (2024). Nginx documentation. [Online].
- Fette, I. and Melnikov, A. (2011). The websocket protocol. Technical Report RFC 6455, IETF.
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine. Doctoral dissertation.
- Grigorik, I. (2013). *High Performance Browser Networking*. O'Reilly Media.
- Jones, M., Bradley, J., and Sakimura, N. (2015). Json web token (jwt). Technical Report RFC 7519, IETF.
- Mahy, R., Matthews, P., and Rosenberg, J. (2010). Traversal using relays around nat (turn). Technical Report RFC 5766, IETF.
- Mongoose (2024). Mongoose odm documentation. [Online].
- M'Raihi, D. et al. (2005). Hotp: An hmac-based one-time password algorithm. Technical Report RFC 4226, IETF.

- M'Raihi, D. et al. (2011). Totp: Time-based one-time password algorithm. Technical Report RFC 6238, IETF.
- OpenJS Foundation (2024a). Express.js documentation. [Online].
- OpenJS Foundation (2024b). Node.js documentation. [Online].
- OWASP Foundation (2021). Multi-factor authentication (mfa) cheat sheet. [Online].
- Redis Ltd. (2024). Redis pub/sub documentation. [Online].
- Rosenberg, J. et al. (2003). Stun – simple traversal of user datagram protocol (udp) through network address translators (nats). Technical Report RFC 3489, IETF.
- Socket.IO (2024). Socket.io documentation. [Online].
- Tailwind Labs (2024). Tailwind css documentation. [Online].
- W3C (2021). Webrtc 1.0: Real-time communication between browsers. [Online].